BÁO CÁO BÀI TẬP LỚN CUỐI KỲ
HỌC PHẦN XỬ LÝ NGÔN NGỮ TỰ NHIÊN – NĂM HỌC 2025 - 2026

Kính gửi:

- Giảng viên phụ trách học phần
- Hội đồng chấm bài tập lớn

I. THÔNG TIN CHUNG

- **Tên đề tài:** Thiết kế RAG framework phân cấp kết hợp Knowledge Graph cho Coding Agent và thử nghiệm với hai bộ Parser trên tài liệu Next.js, Node.js.
- **Nhóm thực hiện:** **[Nhóm ...]** **Lớp học phần:** **[NLP-...]**
- **Giảng viên hướng dẫn:** **[ThS./TS. .....]**
- **Thành viên nhóm và phân công công việc:**

Bảng 1. Danh sách thành viên và phân công công việc

STT	Họ và tên	Mã SV	Lớp	Nhiệm vụ chính	Đóng góp
1	**[Nguyễn Văn A]**	**[2201xxxx]**	**[K16-...]**	Thiết kế Parser interface 5 invariants, Hybrid Retrieval Engine, biên soạn mục III, IV	**[...%]**
2	**[Trần Thị B]**	**[2201xxxx]**	**[K16-...]**	Xây dựng database PostgreSQL/pgvector, 6-tool MCP Server (stdio & Streamable HTTP), hai bộ Parser mẫu, biên soạn mục I, II	**[...%]**
3	**[Lê Văn C]**	**[2201xxxx]**	**[K16-...]**	Xây dựng bộ kiểm thử 100 truy vấn (`TS1`, `TS2`), thực thi Ablation study và kiểm định McNemar, biên soạn mục V, VI	**[...%]**

II. TÓM TẮT

Các hệ thống Retrieval-Augmented Generation truyền thống thường thực hiện chunking theo fixed token window vì giả định người dùng không thể viết riêng một bộ `Parser` cho từng kho tài liệu, dẫn đến làm đứt gãy table và code block, đánh mất ngữ cảnh tiêu đề cha của các mục con trùng tên và làm lệch số thứ tự dòng so với raw text. Với sự phát triển của các `Coding Agent`, ngay cả người dùng không biết lập trình cũng có thể giao cho `Agent` khảo sát kho tài liệu và viết một bộ `Parser` riêng cho miền dữ liệu của mình; điều `Agent` cần là một RAG framework đứng bên ngoài quy định output contract cho `Parser` và cung cấp sẵn toàn bộ tầng storage, retrieval và Knowledge Graph. Báo cáo này trình bày thiết kế của RAG framework

đó cùng hai bộ `Parser` thử nghiệm thực tế. Về mặt kiến trúc, framework định nghĩa `Parser interface` gồm 5 invariants bắt buộc: đường dẫn phân cấp PostgreSQL `ltree`, ánh xạ vị trí dòng 1:1 so với raw text, `contextualized_text` gắn sẵn tiêu đề tổ tiên phục vụ indexing, danh sách tiêu đề cha để tái tạo intermediate node và verbatim citation cho graph edge. Mọi bộ `Parser` tuân thủ 5 invariants này khi lắp vào framework đều sử dụng trực tiếp dual-channel retrieval engine gồm: (1) semantic channel kết hợp Dense Vector `Qwen3-Embedding-0.6B` rút gọn Matryoshka 512 chiều với Sparse Full-Text Search qua Reciprocal Rank Fusion và reranking bằng Cross-Encoder `mmarco-mMiniLMv2-L12-H384-v1`; (2) verbatim search channel dựa trên POSIX regex và trigram index `pg_trgm`. Khi trả kết quả qua 6 MCP tools, framework hợp nhất hai kênh theo Venn partitioning, tách riêng chuỗi tiêu đề cha ra cấp nhóm, sắp xếp các `chunk` cùng nhóm theo đúng reading order từ trên xuống dưới, kiểm soát độ tin cậy qua `Sparse Match Gating`, cho phép `Agent` tự curate edge vào Knowledge Graph với citation grounding nguyên văn và xác thực mọi trích dẫn câu trả lời trực tiếp trên PostgreSQL. Để thử nghiệm framework trên dữ liệu thực tế, hai bộ `Parser` riêng cho tài liệu Next.js MDX và Node.js Markdown được xây dựng và lắp vào hệ thống, indexing 167 tài liệu gồm 162.757 dòng raw text thành 5.633 `chunk` (đạt 100% toàn vẹn ranh giới dòng và khối mã) và đánh giá trên 100 truy vấn thuộc 2 bộ test độc lập (`TS1-Lexical-API` và `TS2-Conceptual`) xây dựng qua quy trình 3 pha cách ly mù. Kết quả thực nghiệm cho thấy cơ chế hợp nhất Dual-Channel Venn giúp tỷ lệ tìm thấy phân đoạn đích đạt 97,0% trên toàn tập (và đạt tuyệt đối 100,0% trên toàn bộ 97/97 truy vấn không bị lọc nhầm phạm vi tài liệu ở đầu vào), đạt Recall@5 = 92,5% (93,0% trên Hybrid RRF), Recall@10 = 95,0%, MRR@10 = 78,9% (trên `TS1-Lexical-API` đạt Hit@1 = 74,0%, Recall@5 = 95,0%, MRR@10 = 83,4%; riêng miền Node.js đạt Recall@5 = 100,0%) và NDCG@10 = 85,0%, vượt trội có ý nghĩa thống kê so với truy xuất từ vựng đơn lẻ theo kiểm định McNemar ($p = 0,0156$).

Từ khóa: Agent-Oriented RAG Framework, Parser Interface, Hierarchical Chunking, Knowledge Graph, Citation Grounding, Hybrid Search, Model Context Protocol.

III. GIỚI THIỆU BÀI TOÁN VÀ CÁC PHƯƠNG PHÁP LIÊN QUAN

1. Bối cảnh và động lực

Các hệ thống Retrieval-Augmented Generation [1] phổ biến hiện nay thường dùng các `Chunker` đa dụng theo fixed token window hoặc recursive character splitting. Lý do của cách tiếp cận này là trước đây, việc đòi hỏi người dùng tự viết một bộ `Parser` riêng cho từng kho tài liệu là rào cản kỹ thuật quá lớn. Tuy nhiên, việc chunking mù quáng mà không hiểu cấu trúc tài liệu gây ra ba hệ quả trực tiếp: (1) cắt ngang table và code block làm hỏng cú pháp [10], [11]; (2) làm mất chuỗi tiêu đề cha của các mục con có tên chung chung như `Options`, `Parameters`, `Examples` khiến embedding model không phân biệt được mục đó thuộc về chủ đề hay hàm nào [12]; và (3) làm mất ánh xạ dòng đối với raw text.

Sự xuất hiện của các LLM và `Coding Agent` đã thay đổi bài toán này: ngay cả người dùng không biết lập trình cũng có thể yêu cầu `Agent` đọc cấu trúc kho tài liệu của họ và viết một bộ

`Parser` riêng cho kho tài liệu đó. Quá trình để `Agent` viết và test `Parser` chỉ cần thực hiện một lần khi đưa miền tài liệu mới vào, sau đó chạy bằng code thuật toán thông thường trên toàn bộ tài liệu mà không tốn chi phí gọi LLM cho từng trang. Tuy nhiên, `Agent` không thể tự xây lại từ đầu toàn bộ database schema, search index và Knowledge Graph cho mỗi dự án. Điều cần thiết là một **RAG framework đứng bên ngoài**, làm nhiệm vụ định nghĩa interface để `Agent` lắp `Parser` vào và cung cấp sẵn toàn bộ backend infrastructure phía sau:

1. **`Parser interface` với 5 output invariants:** Framework loại bỏ hoàn toàn kiểu fixed-window chunking. Thay vào đó, hệ thống định nghĩa một output contract cố định kèm bước kiểm tra tự động: mọi `Parser` muốn lắp vào framework phải trả về các `chunk` có đường dẫn phân cấp `ltree` hợp lệ, line range $[s_i, e_i]$ khớp 1:1 với raw text, văn bản nguyên bản `verbatim_text`, văn bản đã ghép sẵn chuỗi tiêu đề cha `contextualized_text` và mảng `ancestor_titles`.
2. **Tách biệt biểu diễn lúc indexing và lúc retrieval:** Khi nhận đầu ra từ `Parser`, framework tạo embedding vector và FTS index trên trường `contextualized_text` để query khớp được cả tên tài liệu/mục cha lẫn nội dung mục con [12]. Ngược lại, khi trả kết quả retrieval cho `Agent`, framework tự động tách chuỗi tiêu đề cha ra làm thông tin chung của cả nhóm, chỉ trả `verbatim_text` cho từng `chunk` để tránh lặp lại tiêu đề cha nhiều lần, đồng thời sắp xếp các `chunk` trong cùng một nhóm theo reading order tăng dần (s_i ASC) của tài liệu gốc.
3. **Dual-channel retrieval và Knowledge Graph qua MCP:** Phía sau `Parser interface`, framework cung cấp sẵn qua giao thức Model Context Protocol [14] retrieval pipeline kết hợp Hybrid RRF + Cross-Encoder với regex/trigram search channel, cổng `Sparse Match Gating` đánh giá confidence và cơ chế cho phép `Agent` duyệt cũng như tự curate edge vào Knowledge Graph với citation grounding nguyên văn qua trường `citation_text` [1], [13].

2. Phát biểu bài toán

Bài toán thiết kế RAG framework phân cấp và `Parser interface` được phát biểu tổng quát cho mọi miền tài liệu có cấu trúc như sau:

- **Đầu vào của framework:**
  1. Một kho tài liệu có cấu trúc phân cấp $\mathcal{D} = \{d_1, d_2, \ldots, d_M\}$. Mỗi tài liệu d_m có mã định danh phân cấp $\text{slug}(d_m) \in \mathcal{T}_{\text{ltree}}$ và mảng các dòng raw text $L(d_m) = (\ell_1, \ell_2, \ldots, \ell_{|d_m|})$.
  2. Một bộ `Parser` $f_{\text{parse}} : \mathcal{D} \to 2^{\mathcal{C}} \times 2^{\mathcal{E}}$ được `Coding Agent` hoặc lập trình viên viết cho miền tài liệu $\mathcal{D}$ và đăng ký vào `Parser Registry`.
  3. Runtime query gồm câu hỏi ngữ nghĩa q, verbatim pattern hoặc regex p, tập lọc tài liệu $\mathcal{S} \subseteq \mathcal{D}$ và `ltree` path prefix $\pi \in \mathcal{T}_{\text{ltree}}$.
- **Output contract 5 invariants mà `Parser` f_{parse} phải thỏa mãn:** Để framework chấp nhận nạp vào database, mỗi `chunk` $c_i = (\text{path}_i, \text{text}_i^{\text{verb}}, \text{text}_i^{\text{ctx}}, [s_i, e_i], \text{meta}_i) \in \mathcal{C}$ và mỗi graph edge $e \in \mathcal{E}$ do f_{parse} xuất ra phải thỏa mãn 5 invariants:
  1. *Invariant đường dẫn `ltree`*: $\text{path}_i \in \mathcal{T}_{\text{ltree}}$ là duy nhất trong tài liệu, bắt đầu bằng prefix $\text{slug}(d_m)$ và mỗi nhãn phân cấp khớp regex `^[A-Za-z0-9_]{1,255}$`.

2. *Invariant vị trí dòng 1:1*: Chỉ số dòng bắt đầu và kết thúc thỏa mãn $1 \leq s_i \leq e_i \leq |d_m|$, trỏ chính xác vào vị trí dòng trong $L(d_m)$ và không cắt ngang các atomic block như code block hay table.
3. *Invariant tiền tố ngữ cảnh cha*: Trường văn bản ngữ cảnh có dạng chuẩn $\text{text}_i^{\text{ctx}} =$ `"[`context_i`]\n"` $\circ\, \text{text}_i^{\text{verb}}$, trong đó context_i là chuỗi tiêu đề từ gốc tài liệu đến mục hiện tại nối bằng ký hiệu `" > "`.
4. *Invariant danh sách tiêu đề tổ tiên*: Trường meta_i chứa mảng `ancestor_titles` ứng với từng cấp trên path_i để framework tự động tái tạo các virtual parent node khi tree navigation.
5. *Invariant citation grounding trên graph edge*: Mọi cạnh quan hệ $e = (c_u, c_v, r, \tau) \in \mathcal{E}$ phải thuộc tập 9 mã quan hệ chuẩn của hệ thống, thỏa mãn $c_u \neq c_v$ và chứa chuỗi trích dẫn $\tau \neq \emptyset$ xuất hiện nguyên văn trong $\text{text}_u^{\text{verb}}$ sau khi chuẩn hóa khoảng trắng.

– **Đầu ra của framework khi retrieval và navigation:** Từ tập $\mathcal{C}$ và $\mathcal{E}$ đã nạp, với mỗi query (q, p), framework trả về cấu trúc hợp nhất $\mathcal{R}(q, p) = (\mathcal{D}_{\text{hit}}, \mathcal{G}, N_{\text{verb}}, \gamma)$ gồm: danh sách tài liệu deduplicated $\mathcal{D}_{\text{hit}}$; danh sách các context group $\mathcal{G} = (g_1, g_2, \ldots, g_K)$ gom các `chunk` có cùng khóa $(\text{slug}(d_m), \text{context})$ theo thứ tự ưu tiên Venn partitioning ($\mathcal{H}_{\text{sem}} \cap \mathcal{H}_{\text{verb}} \rightarrow \mathcal{H}_{\text{sem}} \setminus \mathcal{H}_{\text{verb}} \rightarrow \mathcal{H}_{\text{verb}} \setminus \mathcal{H}_{\text{sem}}$) nhưng sắp xếp các `chunk` bên trong mỗi nhóm theo natural reading order (s_i ASC, path_i ASC); tổng số verbatim matches N_{verb}; và confidence label $\gamma \in \{\texttt{none}, \texttt{low}, \texttt{medium}, \texttt{high}\}$, cùng các phép duyệt cây $\mathcal{T}_{\text{ltree}}$, duyệt đồ thị $G = (\mathcal{C}, \mathcal{E})$ và đọc raw line slice.

– **Các câu hỏi nghiên cứu trọng tâm:** Từ phát biểu bài toán trên, thực nghiệm của báo cáo tập trung trả lời ba câu hỏi nghiên cứu:

  – **CH1 (Bảo toàn cấu trúc qua `Parser interface` 5 invariants):** Liệu hai bộ `Parser` chuyên biệt cho Next.js MDX và Node.js Markdown lắp vào framework có phân rã trọn vẹn kho tài liệu mà bảo toàn 100% ánh xạ dòng 1:1, tiền tố ngữ cảnh phân cấp và ranh giới khối mã nguồn hay không?

  – **CH2 (Đóng góp của Dual-Channel Venn và giới hạn cửa sổ Reranker):** Sự kết hợp giữa nhánh ngữ nghĩa (Dense + Sparse FTS qua RRF trên $\text{text}_i^{\text{ctx}}$), tầng Cross-Encoder 256 token và kênh nguyên văn regex/trigram đóng góp thế nào vào độ chính xác truy xuất trên hai lớp truy vấn `TS1-Lexical-API` và `TS2-Conceptual`?

  – **CH3 (Vận hành MCP thời gian thực và kiểm chứng trích dẫn):** Kiến trúc Warm Streamable HTTP MCP và cơ chế kiểm chứng đường dẫn `ltree` trực tiếp trên PostgreSQL giải quyết bài toán độ trễ khởi động lạnh và ngăn chặn trích dẫn giả mạo của `Coding Agent` như thế nào?

3. Các phương pháp liên quan

Các công trình nghiên cứu nền tảng liên quan đến hybrid retrieval, structural chunking và Knowledge Graph được chia thành bốn nhóm và tổng hợp tại Bảng 2:

– **Lexical retrieval, Dense retrieval và Two-stage hybrid fusion:** Phương pháp lexical matching như BM25 [2] khớp chính xác các ký hiệu hiếm nhưng không xử lý được query

khác từ vựng bề mặt. Ngược lại, kiến trúc Bi-Encoder như DPR [3] và embedding model dựa trên Decoder-only Transformer như Qwen3-Embedding [4] kết hợp Matryoshka Representation Learning [5] biểu diễn tương đồng ngữ nghĩa trong không gian liên tục $\mathbb{R}^d$. Công thức Reciprocal Rank Fusion [6] cho phép kết hợp thứ hạng từ hai kênh mà không phụ thuộc thang điểm tuyệt đối, trước khi đưa top candidates qua mô hình Cross-Encoder [7], [8] tính cross-attention đầy đủ giữa từng cặp token.

- **Retrieval tài liệu kỹ thuật hỗ trợ lập trình:** DocPrompting [9] chứng minh việc cung cấp đúng đoạn tài liệu chính thức giúp code generation model gọi chính xác các thư viện mới ngoài tập huấn luyện. Tiếp đó, benchmark CodeRAG-Bench [10] chỉ ra rằng tài liệu kỹ thuật chứa nhiều nhiễu định dạng và các phương pháp single-channel retrieval thường xếp hạng sai khi có nhiều hàm mang chữ ký hoặc tiêu đề mục con tương tự nhau.
- **AST chunking và hierarchical contextual retrieval:** Để tránh cắt đứt code block như fixed-window chunking, cAST [11] chunk mã nguồn theo node boundaries trên Abstract Syntax Tree. Ở bước indexing, phương pháp Contextual Retrieval [12] gắn thêm context prefix của tài liệu vào đầu mỗi `chunk` trước khi tạo embedding vector và BM25 index, giúp disambiguate các đoạn văn bản phụ thuộc vào tiêu đề mục cha.
- **Knowledge Graph trong RAG và giao thức MCP cho Agent:** GraphRAG [13] xây dựng entity-relation graph để giải quyết các multi-hop queries, trong khi giao thức Model Context Protocol [14] chuẩn hóa interface để các `Agent` gọi công cụ retrieval và thao tác dữ liệu có cấu trúc.

Bảng 2. Tổng hợp các công trình nghiên cứu liên quan và giới hạn khi áp dụng lên tài liệu phân cấp phức tạp

Công trình	Kỹ thuật cốt lõi	Phạm vi giải quyết chính	Giới hạn trên tài liệu phân cấp và tài liệu kỹ thuật
Robertson & Zaragoza (2009) – BM25 [2]	TF-IDF term weighting với length saturation	Khớp chính xác từ khóa hiếm, tên định danh và mã lỗi	Không xử lý được paraphrased query khác từ vựng bề mặt; standard stemmer làm biến dạng định danh chuyên ngành
Karpukhin et al. (2020) [3]; Zhang et al. (2025) [4]; Kusupati et al. (2022) [5]	Bi-Encoder Dense Embedding và Matryoshka dimension truncation	Biểu diễn ngữ nghĩa câu hỏi tự nhiên và văn bản dài trong không gian $\mathbb{R}^d$	Dễ làm mờ khác biệt nhỏ giữa các định danh gần giống nhau; luôn trả về nearest neighbors ngay cả với out-of-domain queries
Cormack et al. (2009) [6]; Nogueira & Cho (2019) [7]; Wang et al. (2021) [8]	RRF rank fusion kết hợp Cross-Encoder reranking	Cân bằng độ phủ lexical – semantic và tinh chỉnh thứ tự top-K bằng cross-attention	Context window của Cross-Encoder nhỏ gọn thường ngắn hơn các mục tài liệu lớn; thiếu kênh regex matching
Zhou et al. (2023) – DocPrompting [9]; Wang et al. (2025) – CodeRAG-Bench [10]	Retrieval tài liệu API chính thức đưa vào context sinh code	Chứng minh hiệu quả của tài liệu hàm đối với các API mới hoặc ngoài tập huấn luyện	Coi tài liệu là các đoạn văn phẳng có sẵn; chưa giải quyết bài toán làm sạch thẻ JSX, YAML comment hay phân nhánh phiên bản/routing
Zhang et al. (2025) – cAST [11]; Anthropic (2024) – Contextual Retrieval [12]	AST chunking và context prefixing vào đầu chunk khi indexing	Bảo toàn ranh giới khối cú pháp và disambiguate các đoạn con trùng tiêu đề	cAST thiết kế cho tệp mã nguồn thuần túy thay vì tài liệu lai MDX/Markdown; context prefix bị lặp lại dư thừa khi trả nhiều chunk cùng mục
Edge et al. (2024) – GraphRAG [13]	Xây dựng entity-relation graph phục vụ multi-hop queries	Liên kết thông tin phân tán giữa nhiều tài liệu khác nhau	Graph sinh tự động bằng LLM dễ phát sinh hallucinated edges thiếu citation grounding

Technical Gap: Từ Bảng 2, có thể thấy các giải pháp hiện có hoặc dùng generic chunker làm hỏng cấu trúc tài liệu, hoặc hardcode `Parser` vào một định dạng cố định [11], đồng thời dùng LLM sinh context và graph edges cho từng `chunk` gây tốn kém và dễ sinh hallucinated edges [12], [13]. Khoảng trống đặt ra là xây dựng một **RAG framework tách biệt rõ core framework cố định với module `Parser` lắp ghép**: cung cấp output contract 5 invariants để `Coding Agent` chỉ cần viết `Parser` cho miền tài liệu mới, và cung cấp sẵn toàn bộ tầng PostgreSQL đa index, dual-channel Venn retrieval, gom nhóm context theo reading order và Knowledge Graph có citation grounding qua giao thức MCP [14].

4. Đóng góp của báo cáo

Trên cơ sở kế thừa mô hình RRF + Cross-Encoder [6]–[8], kỹ thuật Matryoshka dimension truncation [5] và nguyên lý context prefixing [12], báo cáo thực hiện hai nhóm đóng góp tách biệt:

1. **Thiết kế và hiện thực hóa RAG framework phân cấp độc lập miền dữ liệu:**
   - Định nghĩa `Parser interface` với 5 output invariants, bộ hàm chuẩn hóa đường dẫn `ltree` dùng chung và ingestion pipeline kiểm tra tính toàn vẹn kèm differential vector cache.
   - Xây dựng dual-channel retrieval engine kết hợp Hybrid Search trên vector 512 chiều của `Qwen3-Embedding-0.6B` [4] và FTS cấu hình `simple` có proximity boost qua RRF [6], reranking bằng `mmarco-mMiniLMv2-L12-H384-v1` [8], hợp nhất theo Venn partitioning với kênh tìm kiếm regex/trigram, tách tiêu đề cha ra cấp nhóm theo đúng reading order và gán nhãn confidence qua cổng `Sparse Match Gating`.
   - Xây dựng hệ quản trị Knowledge Graph có hướng cho phép `Agent` duyệt đệ quy và tự curate edge ở runtime dưới ràng buộc kiểm chứng verbatim `citation_text`, tự động mở rộng container target node xuống các leaf nodes, lưu append-only audit log và kiểm chứng trích dẫn câu trả lời qua 6 MCP tools [14] cùng giao diện Web Observatory Dashboard.
2. **Hiện thực hóa hai bộ `Parser` mẫu trên Next.js MDX và Node.js Markdown để kiểm định framework:**
   - Xây dựng hai bộ `Parser` chuyên biệt giải quyết các cấu trúc phức tạp của tài liệu Next.js và Node.js, indexing 167 tài liệu gồm 162.757 dòng raw text thành 5.633 `chunk` bảo toàn 100% code block và vị trí dòng 1:1, đồng thời thiết lập bộ dữ liệu đánh giá 100 truy vấn qua quy trình 3 pha cách ly mù cùng nghiên cứu Ablation và kiểm định thống kê McNemar trên 2 bộ test độc lập (`TS1-Lexical-API` và `TS2-Conceptual`).

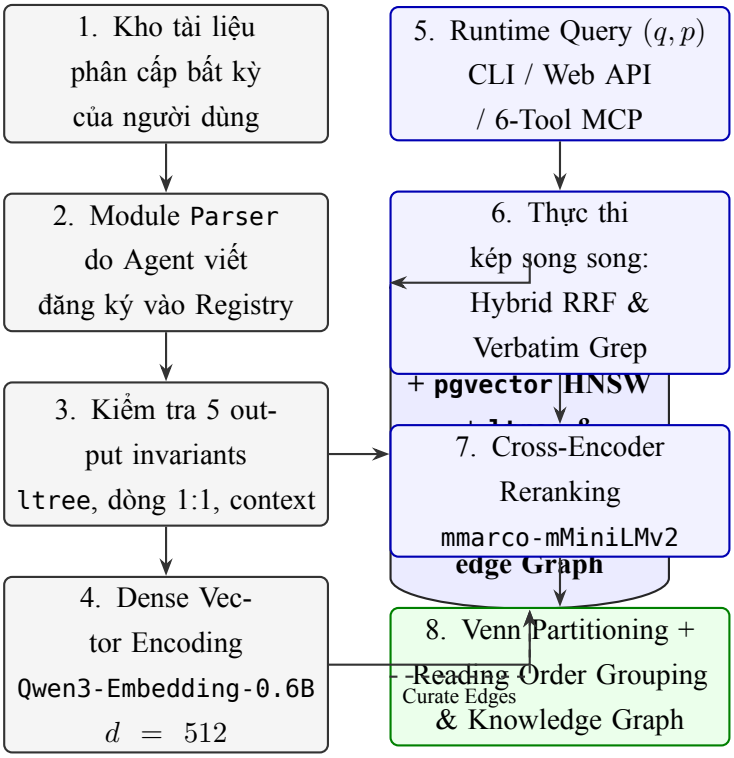
IV. PHƯƠNG PHÁP THỰC HIỆN

Chương này được chia thành hai phần tách biệt: **Phần A từ Mục IV.1 đến Mục IV.4** trình bày kiến trúc của **core RAG framework đứng bên ngoài** — bao gồm `Parser interface` 5 invariants, database schema trên PostgreSQL 16, dual-channel retrieval engine, cơ chế quản trị Knowledge Graph và cơ sở toán học — hoàn toàn không phụ thuộc vào bất kỳ định dạng tài liệu cụ thể nào; **Phần B tại Mục IV.5** trình bày chi tiết **hai bộ `Parser` mẫu cho tài liệu Next.js MDX và Node.js Markdown** được xây dựng để lắp vào framework trên nhằm phục vụ thực nghiệm.

1. Phần A.1 – Kiến trúc tổng thể và Parser interface của framework

Kiến trúc vận hành tổng thể của hệ thống được biểu diễn tại Hình 1, phân tách rõ ranh giới giữa **vùng lắp ghép `Parser` theo 5 output invariants** từ bước 1 đến bước 3 và **lõi storage, hybrid**

retrieval, Knowledge Graph cố định của framework từ bước 4 đến bước 8.



Hình 1. Kiến trúc RAG framework tách biệt `Parser interface` ở bước 1–3 và lõi storage – retrieval cố định ở bước 4–8

Để một `Coding Agent` có thể nhanh chóng viết `Parser` cho một kho tài liệu mới và lắp vào hệ thống, framework cung cấp sẵn `Parser Registry`, bộ hàm chuẩn hóa nhãn cây phân cấp và lớp kiểm tra tự động ở bước 2–3:

- **Điều phối `Parser Registry` qua positive detection:** Mỗi bộ `Parser` khi lắp vào framework chỉ cần định nghĩa hai phương thức: detector method nhận diện tài liệu thuộc miền của mình và parse method phân rã raw text thành cấu trúc `ParsedDocumentDraft`. Nếu một tệp đầu vào không khớp với bất kỳ `Parser` nào đã đăng ký, framework dừng ngay lập tức và báo lỗi định dạng chưa được hỗ trợ thay vì tự ý chạy fixed-window chunking.
- **Bộ hàm chuẩn hóa nhãn và đường dẫn `ltree` dùng chung:** Framework cung cấp sẵn hàm biến đổi mọi tiêu đề mục hoặc đường dẫn tệp từ bất kỳ ngôn ngữ nào về miền nhãn PostgreSQL `ltree` hợp lệ khớp regex `^[A-Za-z0-9_]{1,255}$`: chuyển ký tự `đ/Đ` sang `d/D`, chuẩn hóa Unicode `NFKD` và lược bỏ dấu tổ hợp, thay thế chuỗi ký tự phi chữ-số bằng dấu gạch dưới `_`, cắt tối đa 250 ký tự; các tiêu đề chỉ gồm chữ viết phi-Latinh được ánh xạ sang deterministic MD5 hash 12 ký tự có prefix `u_`.
- **Kiểm tra tự động 5 invariants trước khi ghi database:** Trước khi cho phép bước 4 thực hiện batch encoding, framework kiểm tra mọi `chunk` và graph edge do `Parser` trả về: tính hợp lệ và tính duy nhất của đường dẫn $\text{path}_i \in \mathcal{T}_{\text{ltree}}$, bất đẳng thức dòng $1 \leq s_i \leq e_i \leq |d_m|$, khuôn dạng tiền tố $\text{text}_i^{\text{ctx}} = \texttt{"[context}_i\texttt{]\textbackslash n"} \circ \text{text}_i^{\text{verb}}$, mảng `ancestor_titles` trong metadata và citation grounding của `citation_text` trong `chunk` nguồn.

2. Phần A.2 – Kiến trúc storage, retrieval và Knowledge Graph của framework

a) Các mô-đun xử lý trung tâm và sự đánh đổi kỹ thuật

Sau khi dữ liệu đi qua bước kiểm tra 5 invariants, toàn bộ quá trình storage, retrieval và navigation được thực thi bởi 4 mô-đun cố định của framework:

1. **Dense Bi-Encoder `Qwen/Qwen3-Embedding-0.6B` [4]:** Encode chuỗi $\text{text}_i^{\text{ctx}}$ với độ dài tối đa 2.048 token, áp dụng Matryoshka dimension truncation [5] về số chiều $d = 512$ và chuẩn hóa L_2 ($\|\mathbf{v}\|_2 = 1$).
   - *Căn cứ kỹ thuật & đánh đổi*: Do `Qwen3-Embedding-0.6B` dùng kiến trúc Decoder-only Transformer lấy biểu diễn qua last-token pooling, tokenizer bắt buộc dùng `left-padding` để các pad token không làm lệch hidden state cuối chuỗi. Việc rút gọn Matryoshka xuống 512 chiều giúp giảm 50% dung lượng lưu trữ vector so với 1.024 chiều gốc và tăng tốc độ duyệt HNSW index trong `pgvector` [4], [5].
2. **Kênh Sparse FTS & Kênh Verbatim / Regex Matching:**
   - *Kênh Sparse FTS*: Tạo GIN index trên cột `tsv_content` được cập nhật tự động qua database trigger từ trường $\text{text}_i^{\text{ctx}}$ bằng cấu hình từ điển `simple`. *Đánh đổi kỹ thuật*: từ điển `english` dùng Porter stemmer sẽ cắt biến dạng các thuật ngữ và tên hàm chuyên ngành; từ điển `simple` giữ nguyên vẹn hình thái token theo khoảng trắng và dấu câu, đổi lại không tự động lemmatization tiếng Anh và không hỗ trợ `camelCase` sub-token splitting.
   - *Kênh Verbatim và Regex Matching*: Tìm kiếm trực tiếp bằng POSIX regex (`~` `*~`) hoặc substring (`LIKE`/`ILIKE`) kết hợp toán tử ngưỡng trigram `%` và hàm `word_similarity` của tiện ích `pg_trgm` trên cả $\text{text}_i^{\text{verb}}$ và $\text{text}_i^{\text{ctx}}$. Kênh này bù đắp trực tiếp cho FTS `simple` khi cần tìm chuỗi con nằm giữa một compound identifier, mã lỗi hoặc CLI flag.
3. **Cross-Encoder Reranker `mmarco-mMiniLMv2-L12-H384-v1` [8]:** Gồm 12 lớp Transformer với hidden size $H = 384$, context window tối đa $L_{\text{max}} = 256$ token, nhận cặp $(\text{query}, \text{text}_i^{\text{ctx}})$ để thực hiện cross-attention scoring ở chế độ `CUDA FP16` kèm LRU cache lưu 2.048 cặp gần nhất. *Đánh đổi kỹ thuật*: context window 256 token giữ reranking latency nhỏ, nhưng với các `chunk` dài hơn 256 token, điểm Cross-Encoder chỉ phản ánh mức độ khớp trên chuỗi tiêu đề cha, tiêu đề mục và đoạn mở đầu, còn phần thân phía dưới dựa vào điểm RRF từ Dense Vector 2.048 token và FTS.
4. **Tree Navigation trên `ltree`, Knowledge Graph và Grounded Answer Verification qua 6 MCP tools [14]:**
   - *Tree navigation và tự động tái tạo virtual parent node*: Hỗ trợ 4 hướng duyệt dựa trên toán tử `ltree` gồm `CHILDREN` (`c.path <@ anchor` và `nlevel(c.path) = nlevel(anchor) + 1`), `SUBTREE` (`c.path <@ anchor`), `PARENT_CHAIN` (`c.path @> anchor`) và `SIBLINGS`. Khi một mục cha trong tài liệu chỉ có dòng tiêu đề mà không có đoạn văn thân bài riêng nên không tạo `chunk` vật lý trong bảng `chunks`, truy vấn `PARENT_CHAIN` tự động tái tạo virtual parent node từ mảng `ancestor_titles` lưu trong metadata của nút con với deterministic UUIDv5, đảm bảo cây điều hướng không bị đứt

quãng.

– *Duyệt, curate Knowledge Graph và xác thực trích dẫn câu trả lời*: Hỗ trợ 9 loại quan hệ ngữ nghĩa, trong đó `CONTRADICTS` và `SEE_ALSO` là hai quan hệ đối xứng được recursive CTE `WITH RECURSIVE` tự động duyệt hai chiều kèm cycle detection. Khi `Agent` gọi công cụ thêm hoặc xóa cạnh trỏ tới một container node, framework tự động mở rộng đích xuống toàn bộ các leaf nodes (`c.path <@ target_path`), kiểm tra citation grounding của chuỗi `citation_text` trong `chunk` nguồn và ghi vào bảng append-only audit log trong một atomic transaction. Khi kết thúc điều tra, công cụ `submit_grounded_answer` kiểm tra từng trích dẫn `(path, start_line, end_line)` trực tiếp trên PostgreSQL trước khi chấp nhận câu trả lời của `Agent`.

b) Database Schema trên PostgreSQL 16

Framework lưu trữ toàn bộ dữ liệu trong database schema gồm 5 bảng quan hệ cố định trên PostgreSQL 16 cùng hệ thống index tại Bảng 3.

Bảng 3. Database schema 5 bảng trên PostgreSQL 16 và hệ thống index của framework

Bảng	Các cột và ràng buộc chính	Hệ thống Index & Trigger
`documents`	`id` (UUID PK), `doc_slug` (UNIQUE), `title`, `raw_text`, `metadata` (JSONB)	B-Tree trên `doc_slug`; GIN `jsonb_path_ops` trên `metadata`
`chunks`	`id` (UUID PK), `document_id` (FK), `path` (`LTREE` UNIQUE), `verbatim_text`, `contextualized_text`, `start_line`, `end_line`, `embedding` (`VECTOR(512)`), `tsv_content` (`TSVECTOR`)	HNSW Cosine `m=16, ef_construction=64` trên `embedding`; GiST & B-Tree trên `path`; GIN trên `tsv_content`; 2 GIN index `gin_trgm_ops` trên `verbatim_text` và `contextualized_text`; Trigger tự động cập nhật `tsv_content`
`relation_types`	`code` (PK), `description`, `is_symmetric` (BOOLEAN)	Khởi tạo sẵn 9 loại quan hệ; `CONTRADICTS` và `SEE_ALSO` có `is_symmetric = TRUE`
`graph_edges`	`id` (PK), `source_chunk_id` (FK), `target_chunk_id` (FK), `relation_type` (FK), `citation_text` (NOT NULL), `rationale`	Ràng buộc `CHECK (target != source)` chống tự nối, `UNIQUE (source, target, relation)`; B-Tree trên `source`, `target`, `relation`
`graph_edge_log`	`id` (PK), `action` (`ADDED`/`REMOVED`), `source_path`, `target_path`, `citation_text`, `actor`, `reason`, `created_at`	Append-only audit log; GiST trên `source_path`, `target_path`; B-Tree giảm dần trên `created_at`

Phân định phần kế thừa và tự phát triển: Hệ thống kế thừa trọng số huấn luyện sẵn của mô hình `Qwen3-Embedding-0.6B` [4] và `mmarco-mMiniLMv2-L12-H384-v1` [8], công thức RRF [6], nguyên lý context prefixing [12], thư viện `markdown-it-py` và các tiện ích `pgvector`, `ltree`, `pg_trgm` của PostgreSQL. Các thành phần do nhóm trực tiếp thiết kế và cài đặt gồm: `Parser interface` 5 invariants, ingestion pipeline có differential vector cache, database schema 5 bảng cùng các stored procedures cho hybrid retrieval, verbatim matching và recursive graph traversal, thuật toán Venn partitioning kèm reading order grouping và cổng `Sparse Match Gating`, dịch vụ citation grounding cho Knowledge Graph, 6-tool MCP server (hỗ trợ

cả `stdio` và Warm Streamable HTTP `POST /api/mcp` chia sẻ GPU engine), giao diện Web Observatory Dashboard và hai bộ `Parser` mẫu cho Next.js, Node.js.

3. Phần A.3 – Luồng vận hành dữ liệu trong framework

a) Ingestion pipeline qua Parser và differential vector cache

Khi nạp một tài liệu d_m vào framework:

- **Bước 1 – Gọi `Parser` và kiểm tra 5 invariants:** Tài liệu nguồn được chuyển cho `Parser` tương ứng để tách thành `raw_text`, danh sách N_m `chunk` kèm đường dẫn $\text{path}_i \in \mathcal{T}_{\text{ltree}}$ và danh sách các graph edges tĩnh, sau đó đi qua bước kiểm tra 5 invariants.
- **Bước 2 – Đối chiếu differential vector cache & dọn dẹp `chunk` cũ:** Trước khi gọi GPU, framework truy vấn các `chunk` hiện có của tài liệu trong database theo ánh xạ $\text{path}_i \mapsto (\text{text}_i^{\text{ctx}}, \mathbf{v}_i)$. Các `chunk` có đường dẫn và chuỗi $\text{text}_i^{\text{ctx}}$ không thay đổi được giữ nguyên vector $\mathbf{v}_i \in \mathbb{R}^{512}$ đã lưu; các đường dẫn cũ không còn tồn tại trong bản phân tích mới thuộc tập $\mathcal{P}_{\text{old}} \setminus \mathcal{P}_{\text{new}}$ bị xóa tự động khỏi bảng `chunks`.
- **Bước 3 – Batch encoding kèm cơ chế fallback batch size khi CUDA OOM:** Chỉ tập gồm B_{un} `chunk` mới hoặc có thay đổi nội dung mới được đưa vào `Qwen3-Embedding-0.6B` theo batch size 16 với chiều dài tối đa $L_{\text{emb}} \leq 2048$ để sinh embedding matrix $\mathbf{E} \in \mathbb{R}^{B_{\text{un}} \times 512}$. Nếu xảy ra lỗi CUDA OOM, hệ thống tự động giải phóng CUDA cache và hạ batch size xuống 1 để hoàn tất quá trình nạp.

b) Dual-channel retrieval, tách ngữ cảnh cha và sắp xếp theo reading order

Khi xử lý một query (q, p) với giới hạn kết quả $K = 10$ và reranking candidate pool $P = 10$:

- **Bước 1 – Thực thi song song hai kênh retrieval:**
  1. *Semantic channel*: Encode câu hỏi q thành vector $\mathbf{q} \in \mathbb{R}^{512}$ ($\|\mathbf{q}\|_2 = 1$). Stored procedure trong database lấy ngưỡng candidate pool $C = \max(6P, 120)$, tìm top-C ứng viên Dense theo khoảng cách Cosine và lọc bể $2C \to$ top-C ứng viên Sparse theo điểm lexical, nối qua `FULL OUTER JOIN` và trả về top-P ứng viên có điểm RRF cao nhất. Khi bật Reranker, P cặp $(q, \text{text}_i^{\text{ctx}})$ được cắt ở $L_{\max} = 256$ tạo tensor $[P \times 256]$, tra cứu qua LRU cache và chấm điểm trên Cross-Encoder theo batch size 32 để sinh score vector $\mathbf{s}_{\text{ce}} \in \mathbb{R}^P$, rồi sắp xếp lại lấy top-K kết quả $\mathcal{H}_{\text{sem}}$.
  2. *Verbatim channel*: Thực thi truy vấn đối sánh mẫu p bằng regex hoặc substring kết hợp độ tương đồng trigram trên database, trả về top-K `chunk` khớp mẫu $\mathcal{H}_{\text{verb}}$ cùng tổng số verbatim matches toàn cục N_{verb}.
- **Bước 2 – Venn partitioning và tách chuỗi tiêu đề cha khỏi văn bản `chunk`:** Dựa trên khóa đường dẫn path_i, các kết quả được xếp theo thứ tự ưu tiên ba phân vùng: (1) $\mathcal{H}_{\text{sem}} \cap \mathcal{H}_{\text{verb}}$ tương ứng nhãn `both`, (2) $\mathcal{H}_{\text{sem}} \setminus \mathcal{H}_{\text{verb}}$ tương ứng nhãn `semantic_only`, và (3) $\mathcal{H}_{\text{verb}} \setminus \mathcal{H}_{\text{sem}}$ tương ứng nhãn `verbatim_only` để gán thứ hạng toàn cục rank $\in \{1, \ldots, |\mathcal{H}_{\text{sem}} \cup \mathcal{H}_{\text{verb}}|\}$. Tại mỗi `chunk`, framework tách chuỗi $\text{text}_i^{\text{ctx}} = \texttt{"[context]\n"} \circ \text{text}_i^{\text{verb}}$ ngược lại thành cặp $(\text{context}, \text{text}_i^{\text{verb}})$ và xóa trường `ancestor_titles` dư thừa khỏi metadata trả về (Bảng 4).

– **Bước 3 – Gom nhóm theo ngữ cảnh cha và sắp xếp theo reading order:** Các `chunk` có cùng khóa (`doc_slug`, context) được gom vào một nhóm `ProvisionGroup` g_k dùng chung một dòng tiêu đề cha context duy nhất. Thứ tự giữa các nhóm $g_1, g_2, \ldots$ được giữ theo thứ hạng tốt nhất của `chunk` trong nhóm, nhưng các `chunk` bên trong cùng một nhóm được sắp xếp lại theo reading order (s_i ASC, path$_i$ ASC) để văn bản luôn liền mạch từ dòng trên xuống dòng dưới như trong tài liệu gốc.

Bảng 4. Đối chiếu biểu diễn `verbatim_text` và `contextualized_text` trên một phân đoạn thực tế của tài liệu Next.js (`nextjs.01_getting_started.15_route_handlers`, dòng 87–144)

Trường biểu diễn	Nội dung lưu trữ và vai trò trong pipeline
`verbatim_text` *(Trả về cho Agent trong `ProvisionGroup`)*	`#### With Cache Components\n\nBy default, Route Handlers are dynamic. To cache a GET Route Handler with Cache Components, extract the data fetching into a helper function marked with 'use cache'...` → Giữ nguyên văn bản gốc khớp dòng 87–144; nếu chỉ nhúng trường này, tiêu đề con `With Cache Components` bị trùng lắp với hàng chục mục cùng tên trong các tài liệu khác.
`contextualized_text` *(Đưa vào `Qwen3-0.6B`, FTS `simple` và Reranker)*	`[Route Handlers > Route Handlers > Caching > With Cache Components]\n#### With Cache Components\n\nBy default, Route Handlers are dynamic...` → Tiền tố chuỗi tiêu đề cha giúp cả Dense Vector lẫn FTS `simple` phân biệt chính xác mục `Caching` của `Route Handlers` với mục `Caching` của `Server Components` (câu `e017`).

4. Phần A.4 – Cơ sở toán học của các mô-đun lõi

a) Biểu diễn Dense Bi-Encoder với Matryoshka Representation Learning

Query q và văn bản ngữ cảnh $\text{text}_c^{\text{ctx}}$ của `chunk` c được ánh xạ qua mạng Transformer f_θ [4] thành hidden vector $\tilde{\mathbf{q}}, \tilde{\mathbf{v}}_c \in \mathbb{R}^{D_{\text{full}}}$. Nhờ hàm loss đa chiều lồng nhau của Matryoshka Representation Learning [5], embedding vector $d = 512$ chiều được trích xuất bằng phép cắt chiều và chuẩn hóa L_2:

$$\mathbf{v}_c = \frac{\tilde{\mathbf{v}}_c[1:d]}{\|\tilde{\mathbf{v}}_c[1:d]\|_2} \in \mathbb{R}^{512}, \qquad s_{\text{dense}}(q,c) = 1 - d_{\cos}(\mathbf{q}, \mathbf{v}_c) = \mathbf{q}^\top \mathbf{v}_c \tag{1}$$

trong đó $d_{\cos}(\mathbf{q}, \mathbf{v}_c)$ được tính qua toán tử `<=>` trên HNSW index với độ phức tạp xấp xỉ $\mathcal{O}(\log N)$, xác định Dense rank $r_{\text{dense}}(c) \in \{1, 2, \ldots, C\}$.

b) Proximity-Boosted Sparse Scoring và Trigram Word Similarity

Trên kênh Sparse FTS, gọi $L_q = (w_1, w_2, \ldots, w_m)$ là chuỗi tokenized lexemes của query sau khi tách từ bằng cấu hình `simple`. Hệ thống xây dựng đồng thời AND query $T_{\text{all}} = \bigwedge_{i=1}^{m} w_i$ và phrase proximity query $T_{\text{prox}} = \bigvee_{i=1}^{m-1} (w_i \leftrightarrow w_{i+1})$, với $T_{\text{prox}} = w_1$ khi $m = 1$. Điểm Sparse

của chunk c có vector từ vựng D_c được tính bởi:

$$s_{\text{sparse}}(q, c) = 4.0 \cdot \text{ts_rank}(D_c, T_{\text{prox}}, 32) + 2.0 \cdot \mathbb{I}(D_c \models T_{\text{all}}) \tag{2}$$

với hệ số chuẩn hóa 32 đưa ts_rank về miền $[0, 1)$ theo tỷ số rank/(rank + 1), xác định Sparse rank $r_{\text{sparse}}(c)$.

Trên kênh verbatim matching với mẫu p, gọi $\mathcal{T}(s)$ là tập các trigram gồm các chuỗi 3 ký tự liên tiếp có đệm khoảng trắng đầu/cuối từ của chuỗi s. Độ tương đồng lexical của chunk c đối với mẫu p được tính bằng cực đại hàm `word_similarity` trên cả văn bản nguyên gốc và văn bản ngữ cảnh:

$$s_{\text{verb}}(p, c) = \max_{t \in \{\text{text}_c^{\text{verb}},\, \text{text}_c^{\text{ctx}}\}} \max_{w \sqsubseteq t} \frac{|\mathcal{T}(p) \cap \mathcal{T}(w)|}{|\mathcal{T}(p) \cup \mathcal{T}(w)|} \tag{3}$$

trong đó $w \sqsubseteq t$ là chuỗi con gồm các từ liên tiếp trong t có độ tương đồng Jaccard trigram lớn nhất với p.

c) Reciprocal Rank Fusion và Cross-Encoder Score Blending

Điểm hợp nhất RRF [6] giữa nhánh Dense và nhánh Sparse được định nghĩa là:

$$\text{RRF}(q, c) = \frac{\mathbb{I}(c \in \mathcal{R}_{\text{dense}})}{k_{\text{rrf}} + r_{\text{dense}}(c)} + \frac{\mathbb{I}(c \in \mathcal{R}_{\text{sparse}})}{k_{\text{rrf}} + r_{\text{sparse}}(c)}, \quad k_{\text{rrf}} = 60 \tag{4}$$

Mô hình Cross-Encoder [7], [8] đưa chuỗi ghép nối trực tiếp qua toàn bộ các cross-attention layers để sinh relevance logit $s_{\text{ce}}(q, c) = \mathbf{w}^\top \text{Transformer}(\texttt{[CLS]} \circ q \circ \texttt{[SEP]} \circ \text{text}_c^{\text{ctx}} \circ \texttt{[SEP]})$. Với blending coefficient $\alpha \in [0, 1]$ có giá trị mặc định $\alpha = 1.0$, gọi $r_{\text{ce}}(c)$ là thứ hạng theo $s_{\text{ce}}(q, c)$ và $r_{\text{init}}(c)$ là thứ hạng theo RRF, final ranking score là:

$$S_{\text{final}}(q, c) = \begin{cases} s_{\text{ce}}(q, c), & \text{nếu } \alpha = 1.0 \\ \dfrac{\alpha}{60 + r_{\text{ce}}(c)} + \dfrac{1 - \alpha}{60 + r_{\text{init}}(c)}, & \text{nếu } 0 \le \alpha < 1.0 \end{cases} \tag{5}$$

d) Cơ chế Sparse Match Gating và Citation Grounding trên Knowledge Graph

Gọi has_sparse $= \exists c \in \mathcal{H}_{\text{sem}} : r_{\text{sparse}}(c) < 999$ là biến chỉ thị có ít nhất một kết quả khớp lexical, $s_{\max} = \max_{c \in \mathcal{H}_{\text{sem}}} s_{\text{dense}}(q, c)$, và $\mathcal{S}_{\text{ce}} = \{s_{\text{ce}}(q, c) \mid c \in \mathcal{H}_{\text{sem}}, s_{\text{ce}}(q, c) \neq \varnothing\}$ là tập điểm Cross-Encoder khả dụng. Reranker veto flag được kích hoạt khi veto $= (\mathcal{S}_{\text{ce}} \neq \emptyset) \wedge (\max \mathcal{S}_{\text{ce}} < \tau_{\text{veto}})$.

Hàm confidence $\gamma(\mathcal{H}_{\text{sem}})$ được xác định bởi:

$$\gamma(\mathcal{H}_{\text{sem}}) = \begin{cases} \texttt{none}, & \text{nếu } \mathcal{H}_{\text{sem}} = \emptyset \text{ hoặc } (\neg\text{has_sparse} \wedge s_{\max} < \tau_{\text{low}}) \\ \texttt{low}, & \text{nếu } (\neg\text{has_sparse} \wedge s_{\max} \geq \tau_{\text{low}}) \text{ hoặc veto} \\ \texttt{high}, & \text{nếu has_sparse} \wedge \neg\text{veto} \wedge (s_{\max} \geq \tau_{\text{high}}) \\ \texttt{medium}, & \text{nếu has_sparse} \wedge \neg\text{veto} \wedge (\tau_{\text{low}} \leq s_{\max} < \tau_{\text{high}}) \\ \texttt{low}, & \text{trong các trường hợp còn lại} \end{cases} \tag{6}$$

với $\tau_{\text{low}} = 0.70$, $\tau_{\text{high}} = 0.82$ và $\tau_{\text{veto}} = -1.0$.

Đối với mỗi cạnh đề xuất $e = (c_u, c_v, r, \tau)$ trong quá trình curate Knowledge Graph, giả sử đoạn trích dẫn τ được tách thành chuỗi các token $(\omega_1, \omega_2, \ldots, \omega_L)$. Hệ thống biên dịch whitespace-invariant regex $P(\tau) = \text{escape}(\omega_1) \circ \texttt{"\textbackslash s+"} \circ \text{escape}(\omega_2) \circ \cdots \circ \texttt{"\textbackslash s+"} \circ \text{escape}(\omega_L)$ và chỉ chấp nhận cạnh e khi tồn tại khoảng vị trí $[a, b)$ thỏa mãn:

$$\text{text}_u^{\text{verb}}[a : b] \sim P(\tau) \quad \wedge \quad c_u \neq c_v \tag{7}$$

5. Phần B – Hai bộ Parser thử nghiệm lắp vào framework: Next.js và Node.js

Để minh họa cách lắp ghép các bộ `Parser` miền cụ thể vào `Parser interface` 5 invariants của Phần A, hai bộ `Parser` độc lập được xây dựng cho hai kho tài liệu kỹ thuật có cấu trúc đánh dấu khác biệt nhau là **Next.js MDX** và **Node.js Markdown**.

a) Bộ Parser demo 1: Phân rã tài liệu Next.js MDX

Bộ `Parser` của Next.js nhận diện tài liệu thuộc miền của mình qua prefix `nextjs` hoặc sự hiện diện của khối YAML frontmatter mở đầu đi kèm các thẻ JSX đặc trưng (`<AppOnly>`, `<PagesOnly>`, `<Image>`, `<Video>`, `<FixCardGrid>`), sau đó thực hiện 4 bước biến đổi để xuất ra các `chunk` thỏa mãn 5 invariants:

1. *YAML Frontmatter extraction & bảo toàn dòng 1:1*: Đọc các trường `title`, `description`, `related` trong khối `---...---` ở đầu tệp, sau đó thay từng dòng frontmatter bằng một dòng chỉ chứa khoảng trắng `" "` để tổng số dòng sau làm sạch bằng đúng số dòng của tệp gốc ($|L_{\text{clean}}| = |L_{\text{raw}}|$).
2. *Routing scope tracking cho `<AppOnly>` / `<PagesOnly>`*: Duyệt từng dòng nằm ngoài khối code fence để gán nhãn phạm vi routing $r(i) \in \{\texttt{app}, \texttt{pages}, \varnothing\}$ cho dòng thứ i. Nếu một mục chứa nội dung pha trộn cả hai phạm vi, `Parser` chèn nhãn chuyển ngữ cảnh nội dòng `**[App Router]**` hoặc `**[Pages Router]**`; nếu toàn bộ mục nằm trong một phạm vi duy nhất, chuỗi tiêu đề cha context_i và đường dẫn path_i được gắn nhãn routing tương ứng.
3. *Code block masking & multiline JSX reverse mapping*: Toàn bộ code block ```` ```...``` ```` và inline code `` `...` `` được thay tạm bằng placeholder kèm đúng số ký tự xuống dòng `\n` của khối gốc. Khi gặp thẻ JSX trải trên nhiều dòng như `<Image .../>`, `<Video .../>` hoặc `<FixCardGrid>...</FixCardGrid>`, `Parser` đặt dòng Markdown tổng hợp tại dòng cuối

cùng của khối JSX và điền các dòng trống vào các dòng phía trên để dòng kết thúc khớp 1:1 với dòng đóng thẻ JSX, đồng thời lưu bảng reverse mapping về dòng mở thẻ JSX ban đầu để khôi phục chính xác dòng bắt đầu s_i. Sau khi chuẩn hóa khoảng trắng trên body text, các code block được hoàn trả nguyên văn giúp giữ 100% indentation.

4. *Overview chunking & Codemod heading merging*: Các đoạn văn mở đầu trước tiêu đề H2 đầu tiên được gom thành `chunk` `overview`. Trong tài liệu hướng dẫn chuyển đổi code tự động, các tiêu đề H4 đóng vai trò câu mô tả và có duy nhất một tiêu đề con H5 chứa tên lệnh codemod được hợp nhất thành một `chunk` đơn lẻ dưới mục H3.

Bảng 5. Minh họa cách bộ `Parser` Next.js biến đổi tài liệu MDX để thỏa mãn invariant dòng 1:1 của framework

Dòng	Tài liệu MDX gốc	Sau bước làm sạch 1:1	Đầu ra lắp vào framework
1–4	`---\ntitle: Layouts\n---`	4 dòng trống `" "`	Đưa tiêu đề `"Layouts"` vào metadata tài liệu
5	`<AppOnly>`	1 dòng trống `" "`, đặt trạng thái routing = `app`	Ghi nhận phạm vi `app` cho các dòng tiếp theo
6	`## Creating a layout`	`## Creating a layout`	$\text{path}_i =$ `nextjs....creating_a_layout`, $s_i = 6$
7–9	`<Image alt="Diagram"\n src="/layout.png"\n/>`	Dòng 7–8 trống; dòng 9: `![Diagram](/layout.png)`	Reverse mapping dòng bắt đầu của phần tử từ $9 \to 7$
10–12	```` ```tsx\n export default...\n``` ````	Giữ nguyên 100% indentation qua code block masking	Kết thúc chunk tại $e_i = 12$, ghép `[Layouts > Creating a layout]` vào $\text{text}_i^{\text{ctx}}$

b) Bộ Parser demo 2: Phân rã tài liệu Node.js Markdown

Bộ `Parser` của Node.js nhận diện tài liệu qua prefix `nodejs` hoặc sự hiện diện của comment cấu trúc `<!-- YAML / introduced_in= / source_link= -->` và khối trích dẫn `> Stability:`, sau đó thực hiện 4 bước xử lý chuyên biệt:

1. *Version metadata aggregation từ HTML/YAML comments*: Sau khi thực hiện code block masking, `Parser` quét các khối comment `<!-- YAML added: ... deprecated: ... -->` để tổng hợp dòng thông tin `*Added in: ... --- Deprecated in: ...*` ngay tại dòng mở đầu của comment và thay các dòng còn lại bằng dòng trống nhằm giữ nguyên tổng số dòng, đồng thời xóa các empty anchor tags `<a id="..."></a>` để chúng không bị ghép lệch vào cuối mục đứng trước.
2. *Gộp overloaded function signatures*: Khi gặp các tiêu đề cùng cấp liên tiếp mà mục đứng trước chưa có nội dung thân bài, `Parser` trích xuất tên hàm từ tiêu đề; nếu hai tiêu đề liên tiếp biểu diễn các overloaded signatures của cùng một hàm như `fs.open`, chúng được gộp chung vào cùng một `chunk` và kế thừa dòng bắt đầu s_i của chữ ký đầu tiên.
3. *Phân biệt parameter subheadings nội bộ với standalone sub-APIs*: Khi đang đứng ở một leaf node biểu diễn hàm, phương thức hoặc sự kiện, nếu gặp một tiêu đề con cấp

thấp hơn như H4/H5 dưới H3, thuật toán trích xuất tập tên tham số $\mathcal{P}_{\text{parent}}$ bên trong cặp ngoặc đơn của chữ ký hàm cha. Nếu tiêu đề con chỉ mô tả một tham số hoặc thuộc tính con của tham số thuộc $\mathcal{P}_{\text{parent}}$ — ví dụ tiêu đề `` `options.detached` `` nằm dưới `child_process.spawn(command[, args][, options])` có prefix `options` $\in \mathcal{P}_{\text{parent}}$ — hoặc là mục ghi chú/ví dụ nội bộ, tiêu đề đó được giữ nguyên bên trong phần thân của `chunk` hàm cha. Ngược lại, nếu tiêu đề con là một standalone sub-API như `` `util.inspect.custom` `` dưới `util.inspect(object[, options])` hay các mục bắt đầu bằng `Class:`, `Event:`, `Static method:`, `ERR_*`, `Parser` tách nó thành một nút con riêng trên cây `ltree`.

4. *Cắt tỉa intermediate nodes chỉ chứa metadata & lan truyền chỉ số Stability*: Phép post-order traversal trên cây AST lan truyền chỉ số `Stability` từ mục cha xuống các mục con chưa có nhãn riêng, đồng thời loại bỏ các khối văn bản ở intermediate node chỉ chứa dòng thông báo `Stability` hoặc `Added in` nhằm tránh sinh ra các `chunk` rỗng không mang nội dung giải thích.

V. THỰC NGHIỆM

1. Dữ liệu thực nghiệm từ hai bộ Parser mẫu

Để kiểm định RAG framework sau khi lắp hai bộ `Parser` mẫu ở Mục IV.5, thực nghiệm được tiến hành trên toàn bộ hai kho tài liệu kỹ thuật chính thức: **Next.js Documentation** ở định dạng `.mdx`, `.md` và **Node.js Official API Documentation** ở định dạng `.md`. Thống kê quy mô và phân phối cấu trúc của 167 tài liệu gồm 162.757 dòng raw text, sinh ra 5.633 `chunk` sau khi chạy qua hai bộ `Parser` được trình bày tại Bảng 6 và Bảng 7.

Bảng 6. Thống kê kho ngữ liệu thực nghiệm sau khi indexing qua hai bộ `Parser` Next.js và Node.js

Miền tài liệu	Định dạng	Tài liệu	Total Lines	Chunks	With Code	With Table	Scope & Stability Distribution
Next.js Docs	`.mdx, .md`	101	39.009	1.239	671 (54,16%)	106 (8,56%)	Shared: 1.119; App Router: 95; Pages Router: 25
Node.js API	`.md`	66	123.748	4.394	1.436 (32,68%)	685 (15,59%)	Stability 2: 2.974; Stability 1.x: 389; Stability 0: 161; Stability 3: 33; Khác: 837
Tổng cộng	–	**167**	**162.757**	**5.633**	**2.107 (37,40%)**	**791 (14,04%)**	Chuẩn hóa `ltree` & bảo toàn dòng 1:1

Bảng 7. Phân phối độ dài ký tự, số dòng và độ sâu cây phân cấp `ltree` của 5.633 chunk

Miền tài liệu	Độ dài text^{verb} tính theo ký tự					Lines / Chunk		Độ sâu `nlevel`
	Min	Median	Mean	P95	Max	Median	Mean / Max	Min / Med / Mean / Max
Next.js: 1.239 chunks	60	890	1.185,6	3.112	8.787	18	29,5 / 325	3 / 5 / 4,80 / 7
Node.js: 4.394 chunks	40	356	729,7	2.518	15.726	16	26,3 / 637	2 / 4 / 4,08 / 6

Số liệu tại Bảng 7 phản ánh đặc trưng cấu trúc của chiến lược AST section chunking: median chunk length khá gọn với 890 ký tự trên Next.js và 356 ký tự trên Node.js, nhưng phân phối có dạng heavy-tailed với P95 tương ứng là 3.112 và 2.518 ký tự, và cực đại lên tới 15.726 ký tự ở các mục đặc tả table lớn hoặc ví dụ code đầy đủ.

Để đánh giá khách quan năng lực truy xuất của công cụ `hybrid_search` mà không bị thiên lệch do người viết mã tự đặt câu hỏi nhìn trước cơ sở dữ liệu, bộ dữ liệu kiểm thử `data/eval/retrieval.jsonl` gồm 100 ca kiểm thử được xây dựng qua quy trình **3 pha cách ly mù (blind-isolated phases)** với 6 tác tử độc lập:

1. **Pha 1 – Sinh câu hỏi từ đề cương chủ đề (2 tác tử độc lập):** Tác tử chỉ được đọc danh mục tiêu đề của 167 tài liệu (`corpus_topics_outline.md`), hoàn toàn không được truy cập mã nguồn hay nội dung phân đoạn trong CSDL, để biên soạn câu hỏi cho hai lớp tác vụ:

- **Test set 1 – `TS1-Lexical-API` (`e001`–`e050`):** 50 câu hỏi (25 Next.js, 25 Node.js) tra cứu chính xác tên hàm, chữ ký nạp chồng (overloaded signatures), thuộc tính cấu hình `next.config.js`, mã lỗi hệ thống hoặc CLI flags.
- **Test set 2 – `TS2-Conceptual` (`e051`–`e100`):** 50 câu hỏi (25 Next.js, 25 Node.js) diễn đạt tình huống kỹ thuật tự nhiên về kiến trúc, cơ chế vận hành và các chủ đề trùng tên tiêu đề con giữa các phạm vi phân cấp khác nhau (như *Caching*, *Revalidation*, *Server Actions*, *Streams*, *Cluster*, *TLS Session Resumption*).

2. **Pha 2 – Lập lời gọi `hybrid_search` đầu tiên (2 tác tử độc lập):** Đóng vai `Coding Agent` ở thời điểm gọi công cụ lần đầu, tác tử chỉ được đọc câu hỏi từ Pha 1 và bản đặc tả giao diện MCP của `hybrid_search` (kèm danh mục 167 `doc_slug`) để tự quyết định bộ tham số `query`, `pattern`, `is_regex`, `case_sensitive`, `doc_slugs` và `path_prefix`.
3. **Pha 3 – Đối chiếu CSDL và gán nhãn Ground-Truth (2 tác tử độc lập):** Giữ nguyên tuyệt đối câu hỏi và tham số truy vấn của Pha 1–2, tác tử truy vấn trực tiếp bảng `chunks` trong PostgreSQL để xác định đường dẫn phân cấp `ltree` chuẩn trả lời trực tiếp câu hỏi (`expected`). Tổng cộng có **100 đường dẫn `ltree` đích** được gán nhãn và xác thực tồn tại 100% trong CSDL (Bảng 8).

Bảng 8. Cấu trúc các bộ dữ liệu kiểm thử truy xuất (`data/eval/retrieval.jsonl`)

Test Set	Phạm vi miền	Số câu	Tổng đích	Đích/câu	Mục tiêu kiểm định chính
`TS1-Lexical-API`	Next.js (25) + Node.js (25)	50	50	1,00	Độ chính xác khi tìm ký hiệu API, CLI flags, overloaded signatures và khớp mẫu regex/trigram
`TS2-Conceptual`	Next.js (25) + Node.js (25)	50	50	1,00	Khả năng nắm bắt ngữ nghĩa kiến trúc và phân biệt phạm vi phân cấp `ltree` nhờ context prefix
Tổng cộng	**167 tài liệu (5.633 chunks)**	**100**	**100**	**1,00**	**Đánh giá toàn diện kênh đôi (Dual-Channel Venn) và nghiên cứu Ablation**

2. Thiết lập thực nghiệm và Kiểm thử chức năng

a) Tham số cấu hình của framework và mô hình

Các hyperparameters của framework cùng cơ sở lựa chọn tương ứng được tổng hợp tại Bảng 9.

Bảng 9. Hyperparameters, cấu hình và cơ sở lựa chọn của RAG framework

Tham số / Thành phần	Giá trị thiết lập	Cơ sở lựa chọn
Mô hình Dense Embedding	`Qwen/Qwen3-Embedding-0.6B` [4]	Hỗ trợ đa ngôn ngữ và tài liệu lập trình, vừa bộ nhớ GPU 4 GB ở chế độ FP16
Số chiều Vector d	512 chiều, MRL truncation [5] + chuẩn hóa L_2	Giảm 50% dung lượng lưu trữ so với 1.024 chiều gốc mà vẫn giữ tương đồng Cosine
Vector Index `pgvector`	HNSW `vector_cosine_ops` (`m=16, ef_construction=64`)	Cấu hình mặc định chuẩn của `pgvector` cân bằng tốc độ dựng chỉ mục và độ phủ trên 5.633 chunks
Chuỗi tối đa Embedding	$L_{\text{emb}} = 2.048$ tokens, `left-padding`, batch size = 8–16 (fallback 1)	Bao phủ trọn vẹn P95 độ dài chunk (3.112 ký tự) mà không tràn 4 GB VRAM
Cross-Encoder Reranker	`mmarco-mMiniLMv2-L12-H384` [8], $L_{\max} = 256$, $\alpha = 1,0$, $P = 10$	Mô hình 12 lớp gọn nhẹ giữ độ trễ rerank < 400 ms; LRU cache 2.048 cặp giảm tính toán lặp
Hybrid Search & RRF	$k_{\text{rrf}} = 60$, bể ứng viên $C = \max(6K, 120)$, FTS `simple`	Hằng số $k_{\text{rrf}} = 60$ theo Cormack et al. [6]; từ điển `simple` bảo toàn nguyên vẹn định danh API
Trọng số Sparse FTS	Proximity boost = $4,0$, exact match bonus = $+2,0$	Ưu tiên các phân đoạn chứa cụm từ khóa đứng liền kề và hội đủ toàn bộ token truy vấn
Ngưỡng confidence γ	$\tau_{\text{low}} = 0,70$, $\tau_{\text{high}} = 0,82$, $\tau_{\text{veto}} = -1,0$	Phân tầng độ tin cậy thực nghiệm để kích hoạt cảnh báo khi thiếu bằng chứng khớp từ vựng
Giới hạn MCP & Parser	Nav ≤ 50 nodes; Graph ≤ 100 bước; Raw ≤ 200 dòng; 5 invariants	Chặn tràn cửa sổ ngữ cảnh của `Coding Agent` và bảo toàn 100% ranh giới cú pháp AST

b) Cấu hình phần cứng và môi trường

Môi trường phần cứng và phần mềm phục vụ thực nghiệm được tổng hợp tại Bảng 10.

c) Kiểm thử chức năng và bất biến toàn vẹn dữ liệu

Trước khi đánh giá chất lượng xếp hạng, hệ thống được kiểm tra tự động qua bộ test tích hợp (`uv run scripts/check.py`) và kiểm định trực tiếp trên toàn bộ 5.633 phân đoạn trong CSDL nhằm xác nhận các bất biến kiến trúc (Bảng 11).

3. Evaluation Metrics và Quy tắc chấm điểm

Trên bộ kiểm thử gồm $|Q| = 100$ truy vấn, mỗi truy vấn $q \in Q$ có tập nhãn đích $G_q \subset \mathcal{C}$ gồm đường dẫn phân cấp `ltree` chuẩn. Do tài liệu kỹ thuật có cấu trúc cây phân cấp sâu (tới 7 tầng ở Next.js và 6 tầng ở Node.js), một phân đoạn trả về có đường dẫn p được tính là trả lời đúng cho nhãn đích $e \in G_q$ nếu p trùng khớp hoàn toàn hoặc là nút con trực tiếp nằm trong nhánh của e:

$$\text{answers}(p, G_q) \iff \exists\, e \in G_q : (p = e) \vee \text{startswith}(p,\ e + \text{“.”}) \tag{8}$$

Bảng 10. Cấu hình phần cứng và môi trường thực nghiệm

Thành phần	Thông số kỹ thuật
CPU	AMD Ryzen 5 5500U with Radeon Graphics (6 nhân / 12 luồng xử lý)
GPU	NVIDIA GeForce GTX 1650 (4 GB VRAM, suy luận FP16 trên CUDA)
RAM & Hệ điều hành	22 GiB DDR4; Linux Ubuntu / Debian (x86_64)
Hệ quản trị CSDL	PostgreSQL 16 kèm tiện ích mở rộng `pgvector`, `ltree`, `pg_trgm`
Ngôn ngữ & Thư viện chính	Python 3.13.11 quản lý qua `uv`, PyTorch, Sentence-Transformers, `asyncpg`, `markdown-it-py`, FastAPI
Thời gian nạp & lập chỉ mục	Phân tích AST & kiểm định 5 bất biến (167 tài liệu, 5.633 chunks): $\approx 2,1$ giây; Nhúng lạnh toàn kho (5.633 chunks trên GTX 1650 4 GB): $9.793,4$ giây ($\approx 1,74$ giây/chunk); Nạp ấm qua differential vector cache: $< 3,0$ giây
Độ trễ truy vấn (`p50` / `p95`)	Sparse FTS: $2,8$ ms$/5,6$ ms; Dense Vector: $205,3$ ms$/222,8$ ms; Hybrid RRF: $206,5$ ms$/223,9$ ms; Dual-Channel Venn (không Rerank): $207,6$ ms$/218,2$ ms; Full Dual-Channel Venn (có Rerank): $585,5$ ms$/634,4$ ms
Độ trễ khởi tạo phiên MCP	Subprocess `stdio` lạnh (nạp lại Qwen3 + Reranker): ≈ 29.727 ms ở lệnh đầu; Warm Streamable HTTP (`POST /api/mcp`) dùng chung GPU Engine: ≈ 695–768 ms ở lệnh đầu (giảm $\approx 40\times$)

Bảng 11. Kết quả kiểm thử chức năng và kiểm định bất biến trên kho dữ liệu thực tế

Mã KT	Hạng mục kiểm thử	Phương pháp và phạm vi kiểm chứng	Kết quả
`KT01`	Bất biến ánh xạ dòng 1:1 và bảo toàn code block	Đối chiếu `start_line`, `end_line` của toàn bộ 5.633 chunks với `raw_text` gốc trên 167 tài liệu; kiểm tra cân bằng dấu rào code trên 2.107 chunks chứa code.	Đạt (0 lỗi)
`KT02`	Bất biến đường dẫn `ltree` và tiền tố ngữ cảnh	Kiểm tra biểu thức chính quy nhãn `ltree` và tiền tố `[Tài liệu > Mục cha]` đầu trường `contextualized_text` trên toàn bộ 5.633 chunks.	Đạt (100%)
`KT03`	Cơ chế từ chối định dạng lạ (`CodingParserRegistry`)	Thử nạp tệp nằm ngoài đặc tả `NextJSMDXParser` và `NodeJSMDParser`; xác nhận hệ thống ném ngoại lệ `UnsupportedFrameworkError` tức thì, không dùng generic fallback.	Đạt
`KT04`	Nạp ấm bảo toàn bộ nhớ đệm (`Differential Vector Cache`)	Chạy lại quy trình nạp khi `contextualized_text` không đổi; xác nhận hàm băm MD5 giữ nguyên 5.633 vector 512 chiều mà không gọi lại GPU.	Đạt (< 3s)
`KT05`	Kiểm chứng trích dẫn nguyên văn Knowledge Graph	Gọi `graph_curate_edges` với trích dẫn hợp lệ (mở rộng container node xuống các leaf nodes, ghi `graph_edge_log`) và trích dẫn bịa đặt (bị chặn bởi `E_CITATION_NOT_FOUND`).	Đạt
`KT06`	Duyệt phân cấp `ltree` và trích xuất dòng thô	Kiểm tra `hierarchical_navigate` (tái tạo virtual parent nodes cho mục container) và `get_raw` (chặn biên nghiêm ngặt tối đa 200 dòng/lần).	Đạt
`KT07`	Kiểm chứng trích dẫn câu trả lời (`submit_grounded_answer`)	Xác thực mọi trích dẫn `(path, start_line, end_line)` của Agent trên PostgreSQL trước khi chấp nhận câu trả lời; từ chối ngay khi đường dẫn `ltree` không tồn tại hoặc thiếu trích dẫn.	Đạt

Gọi $\hat{R}_q^{(k)} = (\hat{c}_1, \hat{c}_2, \ldots, \hat{c}_k)$ là danh sách k phân đoạn đầu tiên hệ thống trả về (sắp theo thứ tự ưu tiên giỏ Venn: `both` → `semantic_only` → `verbatim_only`):

- **Tỷ lệ tìm thấy (`Found`) và phân rã giỏ Venn:** Một truy vấn được tính là `Found` nếu tồn tại ít nhất một phân đoạn thỏa mãn Phương trình (8) trong kết quả hợp nhất của `hybrid_search`, đồng thời được phân vào một trong bốn giỏ loại trừ nhau: **Cả hai kênh (`both`)**, **Chỉ ngữ nghĩa (`semantic_only`)**, **Chỉ nguyên văn (`verbatim_only`)** hoặc **Bỏ sót (`missed`)**.
- **Hit@k và Recall@k:**

$$\begin{aligned} \text{Hit@}k &= \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}\Big(\exists\, \hat{c} \in \hat{R}_q^{(k)} : \text{answers}(\hat{c}, G_q) \Big)\,, \\ \text{Recall@}k &= \frac{1}{|Q|} \sum_{q \in Q} \frac{\Big| \Big\{ e \in G_q \mid \exists\, \hat{c} \in \hat{R}_q^{(k)} : \text{answers}(\hat{c}, \{e\}) \Big\} \Big|}{|G_q|} \end{aligned} \tag{9}$$

- **MRR@k – Mean Reciprocal Rank:** Với r_q^* là thứ hạng đầu tiên xuất hiện một phân đoạn thỏa mãn $\text{answers}(\hat{c}_i, G_q)$:

$$\text{MRR@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{r_q^*}, \quad r_q^* = \min \left\{ i \in \{1, \ldots, k\} \mid \text{answers}(\hat{c}_i, G_q) \right\}, \quad \frac{1}{\infty} = 0 \tag{10}$$

- **NDCG@k – Normalized Discounted Cumulative Gain:**

$$\text{NDCG@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{\text{DCG}_q@k}{\text{IDCG}_q@k}, \qquad \text{DCG}_q@k = \sum_{i=1}^{k} \frac{\mathbb{I}_{\text{unique}}(\hat{c}_i, G_q)}{\log_2(i+1)} \tag{11}$$

- **Kiểm định McNemar hai phía chính xác (Exact Two-Sided McNemar Test):** Để kiểm chứng sự khác biệt giữa hai cấu hình A và B trên cùng 100 truy vấn có phải do ngẫu nhiên hay không, gọi b là số câu A đúng nhưng B sai (*Tốt lên*) và c là số câu A sai nhưng B đúng (*Kém đi*). Giá trị p-value nhị thức hai phía chính xác được tính theo:

$$p\text{-value} = \min\left(1,\ 2 \sum_{i=0}^{\min(b,c)} \binom{b+c}{i} 0,5^{b+c} \right) \tag{12}$$

4. Kết quả thực nghiệm trên từng test set

a) Kết quả chi tiết theo test set, miền tài liệu và kênh Venn

Bảng 12 ghi nhận kết quả của cấu hình **Dual-Channel Venn** trên hai bộ test `TS1-Lexical-API` và `TS2-Conceptual`, tách riêng theo từng miền tài liệu `Next.js` và `Node.js`.

Số liệu tại Bảng 12 cho thấy ba đặc trưng quan trọng:

1. **Độ phủ 97,0% trên toàn tập và 100,0% khi phạm vi lọc hợp lệ:** Cả hai biến thể của Dual-Channel Venn đều tìm thấy phân đoạn đích trên **97/100 câu hỏi (Found = 97,0%)**. Đáng chú ý, trên miền `Node.js` (50 câu), tỷ lệ tìm thấy đạt tuyệt đối **100,0%** ở cả hai bộ

Bảng 12. Kết quả truy xuất của cấu hình Dual-Channel Venn theo từng bộ test và miền dữ liệu (%)

Test Set	Miền	Found	Giỏ (B/S/V/M)	Hit@1	Hit@5	R@5	R@10	MRR@10	NDCG@10
TS1-Lexical-API	Next.js (25)	96,0	20 / 3 / 1 / 1	72,0	88,0	90,0	92,0	78,2	84,9
	Node.js (25)	**100,0**	24 / 1 / 0 / 0	76,0	**100,0**	**100,0**	**100,0**	87,3	93,6
	Tổng TS1 (50)	**98,0**	**44 / 4 / 1 / 1**	**74,0**	**94,0**	**95,0**	**96,0**	**82,8**	**89,2**
TS2-Conceptual	Next.js (25)	92,0	19 / 4 / 0 / 2	44,0	88,0	88,0	92,0	63,0	71,8
	Node.js (25)	**100,0**	22 / 2 / 1 / 0	64,0	92,0	92,0	96,0	77,4	83,0
	Tổng TS2 (50)	**96,0**	**41 / 6 / 1 / 2**	**54,0**	**90,0**	**90,0**	**94,0**	**70,2**	**77,4**
Toàn cục (100 câu, có Rerank)		**97,0**	**85 / 10 / 2 / 3**	**64,0**	**92,0**	**92,5**	**95,0**	**76,5**	**83,3**
Toàn cục (100 câu, không Rerank)		**97,0**	**85 / 10 / 2 / 3**	**68,0**	**92,0**	**92,5**	**95,0**	**78,9**	**85,0**

TS1 và TS2. Ba câu duy nhất bị bỏ sót trên Next.js (e011, e070, e074) đều do tác tử Pha 2 truyền bộ lọc cứng doc_slugs chỉ gồm 3 tài liệu ứng viên không chứa tài liệu đích; trên toàn bộ **97/97 câu hỏi mà doc_slugs bao hàm tài liệu đích, hệ thống đạt tỷ lệ tìm thấy 100%**, trong đó **85 câu** khớp đồng thời cả hai kênh (both), **10 câu** tìm thấy qua kênh ngữ nghĩa (semantic_only), và **2 câu** (e021, e083) được cứu sống hoàn toàn nhờ kênh nguyên văn (verbatim_only).

2. **Độ chính xác rất cao trên các truy vấn tra cứu API (TS1-Lexical-API):** Với các câu hỏi nhắm vào tên hàm, chữ ký nạp chồng và cờ cấu hình, hệ thống đạt **Recall@5 = 95,0%**, **Recall@10 = 96,0%**, **Hit@1 = 74,0%**, **MRR@10 = 82,8%–83,4%** và **NDCG@10 = 89,2%–89,9%**. Riêng trên miền Node.js của TS1, hệ thống đạt tuyệt đối **Found = 100,0%, Hit@5 = 100,0%, Recall@5 = 100,0% và Recall@10 = 100,0%**.
3. **Khả năng bao phủ các truy vấn tình huống kiến trúc (TS2-Conceptual):** Dù câu hỏi trong TS2 mô tả hiện tượng kỹ thuật dài dòng mà không nêu trực tiếp tiêu đề mục, hệ thống vẫn đạt **Recall@5 = 90,0%** và **Recall@10 = 94,0%** (trên Node.js đạt **Recall@10 = 96,0%**).

b) Phân bố nhãn độ tin cậy (γ) của cổng Sparse Match Gating

Bảng 13 đối chiếu phân bố nhãn độ tin cậy $\gamma \in \{\texttt{none}, \texttt{low}, \texttt{medium}, \texttt{high}\}$ giữa các cấu hình truy xuất đơn lẻ và cấu hình kết hợp.

5. Đối sánh các cấu hình và Ablation Study

Để tách biệt đóng góp thực tế của từng mô-đun trong framework, nghiên cứu Ablation được thực hiện trên cùng 100 ca kiểm thử qua 6 cấu hình tại Bảng 14:

1. **AST + Sparse FTS Only**: Chỉ xếp hạng bằng điểm từ vựng $s_{\text{sparse}}(q, c)$ ở Phương trình (2) với cấu hình simple.
2. **AST + Dense Vector Only**: Chỉ xếp hạng bằng độ tương đồng Cosine $s_{\text{dense}}(q, c)$ ở Phương trình (1) trên vector 512 chiều Qwen3-Embedding-0.6B.
3. **AST + Hybrid RRF w/o Rerank**: Kết hợp Dense + Sparse trên $\text{text}_c^{\text{ctx}}$ (đã gắn tiền tố ngữ cảnh cha [Tài liệu > Mục cha]) qua công thức RRF ở Phương trình (4) với $k_{\text{rrf}} = 60$,

Bảng 13. Phân bố nhãn độ tin cậy γ qua cổng kiểm soát Sparse Match Gating trên 100 truy vấn

Cấu hình / Tập truy vấn	`none` (%)	`low` (%)	`medium` (%)	`high` (%)	Nhận xét hành vi cổng kiểm soát
1. Chỉ Sparse FTS (`ts_rank`)	3,0	97,0	0,0	0,0	Thiếu tương đồng Cosine ($s_{\max} = 0$) nên kẹp ở mức `low`
2. Chỉ Dense Vector (`Qwen3-0.6B`)	24,0	76,0	0,0	0,0	Thiếu khớp từ vựng (has_sparse = False) nên kẹp ở `low`
6. Full Dual-Channel Venn (`TS1-API`)	0,0	30,0	56,0	14,0	70,0% truy vấn API đạt mức tin cậy `medium` hoặc `high`
6. Full Dual-Channel Venn (`TS2-Concept`)	0,0	26,0	66,0	8,0	74,0% truy vấn khái niệm đạt mức `medium` hoặc `high`
6. Toàn cục (100 truy vấn)	**0,0**	**28,0**	**61,0**	**11,0**	**72,0%–73,0% đạt mức `medium`/`high`; 0% bị gán `none`**

không dùng Cross-Encoder.

4. **AST + Hybrid RRF + Cross-Encoder Rerank**: Bật thêm tầng xếp hạng lại bằng `mmarco-mMiniLMv2` ($L_{\max} = 256$ token, $\alpha = 1,0$) trên $P = 10$ ứng viên đầu tiên của kênh ngữ nghĩa.
5. **Dual-Channel Venn w/o Rerank**: Hợp nhất theo phân hoạch Venn giữa kênh ngữ nghĩa Hybrid RRF (không Rerank) và kênh nguyên văn regex/trigram.
6. **Full Dual-Channel Venn (with Rerank)**: Cấu hình đầy đủ kết hợp kênh ngữ nghĩa đã qua Cross-Encoder Rerank với kênh nguyên văn regex/trigram và gom nhóm theo ngữ cảnh cha.

Bảng 14. Đối sánh các cấu hình Ablation và độ trễ truy vấn trên 100 ca kiểm thử (`data/eval/ablation_results.json`)

Phương pháp / Cấu hình	Found	MRR@10 theo bộ (%)		Toàn cục trên 100 truy vấn (%)					Latency (ms)
	(%)	`TS1-API`	`TS2-Con`	Hit@1	Hit@5	R@5	NDCG@10	MRR@10	p50 / p95
1. AST + Sparse FTS Only	88,0	77,8	67,9	60,0	86,0	86,0	78,6	72,9	2,8 / 5,6
2. AST + Dense Vector Only	95,0	70,6	73,1	59,0	90,0	91,0	79,6	71,9	205,3 / 222,8
3. AST + Hybrid RRF w/o Rerank	95,0	82,8	**75,2**	**68,0**	**93,0**	**93,0**	**85,0**	**79,0**	206,5 / 223,9
4. AST + Hybrid RRF + Rerank	95,0	74,6	66,9	58,0	89,0	89,5	79,0	70,7	584,6 / 619,9
5. Dual-Channel Venn w/o Rerank	**97,0**	**83,4**	74,4	**68,0**	92,0	92,5	**85,0**	78,9	207,6 / 218,2
6. Full Dual-Channel Venn	**97,0**	82,8	70,2	64,0	92,0	92,5	83,3	76,5	585,5 / 634,4

Để kiểm chứng ý nghĩa thống kê của các mức chênh lệch giữa từng cặp cấu hình trong Bảng 14, Bảng 15 trình bày kết quả kiểm định McNemar hai phía chính xác theo Phương trình (12).

Bảng 15. Kiểm định McNemar hai phía chính xác giữa các cặp cấu hình Ablation ($N = 100$)

Cặp đối sánh (Cấu hình A vs. B)	**Chỉ số**	**Tốt lên (b)**	**Kém đi (c)**	**p-value**	**Kết luận thống kê ($\alpha = 0,05$)**
Hybrid RRF (3) vs. Sparse FTS Only (1)	`Found / Hit@5`	7	0	**0,0156**	Có ý nghĩa thống kê ($p < 0,05$)
Hybrid RRF (3) vs. Sparse FTS Only (1)	`Hit@1`	12	4	0,0768	Cải thiện rõ rệt ở biên $\alpha = 0,10$
Hybrid RRF (3) vs. Dense Vector Only (2)	`Hit@1`	17	8	0,1078	Nâng `Hit@1` từ $59,0\%$ lên $68,0\%$ (+9,0 điểm %)
Sparse FTS Only (1) vs. Dense Vector Only (2)	`Found`	2	9	0,0654	Bổ trợ ngược chiều giữa `TS1` và `TS2`
Dual-Channel (6) vs. Hybrid + Rerank (4)	`Hit@1`	10	4	0,1796	Kéo 10 câu bị Reranker hạ bậc về lại hạng 1
Dual-Channel (6) vs. Hybrid RRF (3, 4)	`Found`	2	0	0,5000	Cứu 2 câu `e021`, `e083` nhờ giỏ `verbatim_only`

6. Thảo luận, đánh đổi kiến trúc và phân tích lỗi chi tiết

Từ số liệu thực nghiệm tại Bảng 12–15 và kết quả đối chiếu từng ca truy vấn cụ thể trong tệp `ablation_results.json`, có thể rút ra năm nhận định kỹ thuật cốt lõi:

- **Sự bổ trợ ngược chiều giữa Sparse FTS và Dense Vector:** Khi đứng riêng lẻ, **Sparse FTS (1)** đạt `MRR@10 = 77,8%` trên `TS1-Lexical-API` nhờ khớp chính xác tên định danh, nhưng tụt xuống `67,9%` trên `TS2-Conceptual` và bỏ sót tới 12/100 câu (`Found = 88,0%`) khi câu hỏi mô tả hiện tượng bằng từ vựng khác với tài liệu hoặc khi từ điển `simple` không tách các biểu thức có dấu câu đặc thù. Ngược lại, **Dense Vector (2)** nâng tỷ lệ tìm thấy lên `95,0%` (`Recall@10 = 95,5%`) và đạt `MRR@10 = 73,1%` trên `TS2-Conceptual`, nhưng chỉ đạt `MRR@10 = 70,6%` trên `TS1`. Khi hợp nhất qua **Hybrid RRF (3)**, `Hit@1` tăng từ $60,0\%$ (Sparse) và $59,0\%$ (Dense) lên **68,0%**, `Recall@5` đạt **93,0%**, `MRR@10` tăng lên **79,0%** và `NDCG@10` đạt **85,0%**, vượt trội có ý nghĩa thống kê so với Sparse FTS trên cả `Found` và `Hit@5` (7 câu tốt lên so với 0 câu kém đi, $p = 0,0156$).
- **Sự đánh đổi thực tế giữa bảo toàn atomic AST chunk và cửa sổ 256 token của Cross-Encoder:** Bảng 14 phơi bày một hiện tượng thực nghiệm rất đáng chú ý khi đối chiếu cấu hình **(3) Hybrid RRF w/o Rerank** và **(4) Hybrid RRF + Cross-Encoder Rerank**:
  1. Tầng Cross-Encoder đẩy thành công **10 câu** từ các vị trí bên dưới lên thẳng hạng 1 (`e003`, `e024`, `e036`, `e038`, `e047`, `e051`, `e060`, `e061`, `e085`, `e089`) khi thông tin trả lời nằm gọn trong 256 token đầu tiên của phân đoạn.
  2. Tuy nhiên, nó lại làm tụt hạng 1 của **20 câu khác** (khiến `Hit@1` của riêng nhánh ngữ nghĩa giảm từ $68,0\%$ xuống $58,0\%$, `MRR@10` giảm từ $79,0\%$ xuống $70,7\%$), đồng thời làm độ trễ trung vị `p50` tăng từ $206,5$ ms lên $584,6$ ms. Phân tích các câu bị tụt hạng sâu như câu `e078` (`nodejs.child_process.overview`: tụt từ hạng 1 xuống hạng 9), câu `e068` (tụt từ hạng 1 xuống hạng 8), câu `e029` (tụt từ hạng 1 xuống hạng 7), câu `e012`, `e054`, `e076` (tụt từ hạng 1 xuống hạng 5) hay câu `e086`, `e098` (tụt từ hạng 1 xuống hạng 4) chỉ ra nguyên nhân trực tiếp: do chiến lược AST chunking giữ nguyên vẹn toàn bộ mục tài liệu thành một phân đoạn đơn lẻ (P95 dài tới 2.518–3.112 ký tự), phần giải thích chi tiết trả lời cho câu hỏi thường nằm ở nửa sau của phân đoạn, vượt ra ngoài cửa sổ cắt $L_{\text{max}} = 256$ token của `mmarco-mMiniLMv2`. Khi sử dụng hệ số trộn $\alpha = 1,0$, điểm Cross-Encoder tính trên 256 token đầu đã ghi đè lên thứ hạng RRF vốn được tính trên toàn bộ 2.048 token.
- **Cơ chế tự sửa lỗi và cứu hộ của kênh nguyên văn (Dual-Channel Venn):** Việc hợp nhất song song kênh nguyên văn regex/trigram ở cấu hình **(5, 6)** giải quyết trực tiếp hai điểm yếu trên:
  1. *Cứu sống các truy vấn bị kênh ngữ nghĩa bỏ sót ngoài top 10*: Ở câu `e021` (hỏi về tiền tố thuộc tính nội bộ `$ACTION_` khi truyền đối tượng `FormData` của Next.js Server Action vào hàm `Object.fromEntries`) và câu `e083` (hỏi về phương thức `module.syncBuiltinESMExports()` trong Node.js), từ điển `simple` không khớp được token khiến nhánh Hybrid RRF đẩy mục đích ra ngoài top 10; nhưng kênh nguyên văn với

`pattern = "$ACTION"` và `pattern = "syncBuiltinESMExports"` đã bắt trúng đích trong giỏ `verbatim_only`, nâng `Found` toàn cục từ $95,0\%$ lên **97,0%**.

2. *Kéo ngược các phân đoạn dài bị Reranker hạ bậc về lại hạng 1*: Nhờ quy tắc ưu tiên giỏ giao `both` ($\mathcal{H}_{\text{sem}} \cap \mathcal{H}_{\text{verb}}$) lên trước `semantic_only`, **10 câu hỏi** bị tầng Cross-Encoder làm tụt hạng ở cấu hình (4) gồm `e007` ($2 \to 1$), `e012` ($5 \to 1$), `e018` ($2 \to 1$), `e029` ($7 \to 1$), `e031` ($3 \to 1$), `e035` ($2 \to 1$), `e079` ($2 \to 1$), `e086` ($4 \to 1$), `e096` ($3 \to 1$) và `e098` ($4 \to 1$) đã được kéo thẳng về vị trí số 1 nhờ chứa đúng mẫu `pattern` (`listen(0)`, `lifo`, `cleanup`, `ticketKeys`).

– **Phân tích nguyên nhân gốc rễ của 3 ca bỏ sót (`e011`, `e070`, `e074`):** Cả 3 ca duy nhất có `found = False` trên toàn bộ 100 truy vấn đều không xuất phát từ thuật toán xếp hạng mà do **bộ lọc cứng `doc_slugs` của tác tử Pha 2**: ở câu `e011` (hỏi về ngưỡng `cacheLife`), tác tử chỉ truyền 3 tài liệu `08_caching`, `incremental_static_regeneration_cache_components` và `migrating_to_cache_components` trong khi đặc tả `cacheLife` nằm ở tài liệu `09_revalidating`; tương tự, ở câu `e070` và `e074`, tác tử lọc vào các tài liệu lý thuyết `prefetching` và `rendering_philosophy` trong khi ví dụ thực hành nằm ở `keeping_pages_static` và `package_bundling`. Trong thực tế vận hành nhiều bước của `Coding Agent`, khi lượt gọi đầu tiên với `doc_slugs` hẹp không trả về thông tin cần tìm, `Agent` sẽ bỏ bộ lọc `doc_slugs` ở lượt gọi thứ hai để tìm trên toàn kho.

– **Hiệu năng vận hành thực tế của kiến trúc Warm MCP Server:** Khi tích hợp với các CLI Agent bên ngoài, việc khởi chạy tiến trình con `stdio` mới cho mỗi câu hỏi gây ra độ trễ khởi động lạnh lên tới 29.727 ms ở lệnh gọi `hybrid_search` đầu tiên do phải nạp lại mô hình `Qwen3-Embedding-0.6B` và `CrossEncoderReranker` lên GPU. Bằng cách triển khai giao thức **Warm MCP Streamable HTTP** (`POST /api/mcp`) chia sẻ trực tiếp `RetrievalEngine` đã làm ấm sẵn trên FastAPI cùng cơ chế ưu tiên bộ nhớ đệm cục bộ (`local_files_only=True`), độ trễ lệnh gọi công cụ đầu tiên của `Agent` giảm xuống chỉ còn 695–768 ms, đáp ứng trải nghiệm hỏi đáp thời gian thực.

Tổng hợp trả lời các câu hỏi nghiên cứu (CH1–CH3):

– **Trả lời CH1:** Hai bộ `Parser` mẫu cho Next.js MDX và Node.js Markdown đã phân rã toàn bộ 167 tài liệu (162.757 dòng) thành 5.633 `chunk` trong $\approx 2,1$ giây, đạt **100%** toàn vẹn ánh xạ dòng 1:1 (`line_errors = 0`), **100%** tiền tố ngữ cảnh hợp lệ (`ctx_errors = 0`) và bảo toàn trọn vẹn 2.107 phân đoạn chứa code block (`KT01`, `KT02`).

– **Trả lời CH2:** Kết hợp Dense + Sparse qua RRF trên $\text{text}_i^{\text{ctx}}$ vượt trội có ý nghĩa thống kê so với Sparse FTS đơn lẻ ($p = 0,0156$), nâng `MRR@10` lên **79,0%** và `Recall@5` lên **93,0%**. Việc bổ sung kênh nguyên văn theo phân hoạch Venn cứu sống các truy vấn ký hiệu đặc thù (`e021`, `e083`), nâng tỷ lệ tìm thấy lên **97,0%** (và **100%** trên 97/97 câu có phạm vi lọc hợp lệ), đồng thời kéo 10 phân đoạn dài bị cửa sổ 256 token của Cross-Encoder hạ bậc về lại hạng 1.

– **Trả lời CH3:** Kiến trúc Warm Streamable HTTP MCP (`POST /api/mcp`) giảm độ trễ lệnh gọi công cụ đầu tiên từ 29.727 ms xuống 695–768 ms ($\approx 40\times$), trong khi công cụ

`submit_grounded_answer` đối chiếu trực tiếp từng bộ ba trích dẫn (`path, start_line, end_line`) trên PostgreSQL (`KT07`), chặn đứng mọi trích dẫn đường dẫn `ltree` không tồn tại trong kho ngữ liệu.

VI. KẾT LUẬN

- **Kết quả thực hiện:** Báo cáo đã thiết kế và hiện thực hóa một RAG framework phân cấp kết hợp Knowledge Graph tách biệt rõ core framework cố định với module `Parser` lắp ghép:
  1. Về **core framework đứng bên ngoài**, hệ thống định nghĩa `Parser interface` gồm 5 output invariants, cung cấp tầng storage trên PostgreSQL 16 đa index (`pgvector` HNSW 512 chiều từ `Qwen3-Embedding-0.6B`, `ltree`, FTS `simple`, `pg_trgm`), dual-channel retrieval engine hợp nhất Hybrid RRF + Cross-Encoder `mmarco-mMiniLMv2` với verbatim/regex channel theo Venn partitioning, tự động tách tiêu đề cha ra cấp nhóm và sắp xếp `chunk` theo reading order, kiểm soát độ tin cậy qua `Sparse Match Gating` và cho phép `Agent` duyệt, tự curate Knowledge Graph cũng như xác thực trích dẫn câu trả lời qua 6 MCP tools.
  2. Về **thử nghiệm lắp ghép `Parser`**, hai bộ `Parser` mẫu cho tài liệu Next.js MDX và Node.js Markdown đã được xây dựng và lắp vào framework, indexing thành công 167 tài liệu gồm 162.757 dòng raw text thành 5.633 `chunk` bảo toàn 100% code block và vị trí dòng 1:1. Kết quả đánh giá thực nghiệm trên 100 truy vấn thuộc 2 bộ test độc lập (`TS1-Lexical-API` và `TS2-Conceptual`) đạt tỷ lệ tìm thấy **Found = 97,0%** (**100,0%** trên toàn bộ 97/97 truy vấn có phạm vi lọc tài liệu hợp lệ và **100,0%** trên toàn bộ miền Node.js), **Recall@5 = 92,5%–93,0%** (**95,0%** trên `TS1` và **100,0%** trên Node.js `TS1`), **Recall@10 = 95,0%**, **MRR@10 = 78,9%–79,0%** (**83,4%** trên `TS1`) và **NDCG@10 = 85,0%** với độ trễ truy vấn trung vị $206,5$–$207,6$ ms.
- **Hạn chế kỹ thuật:** (1) Khi đưa một miền tài liệu mới vào framework, quá trình để `Coding Agent` viết và tinh chỉnh `Parser` vẫn cần chạy test lặp lại ở đầu kỳ để xử lý hết các edge cases định dạng và thỏa mãn 5 output invariants; (2) Chiến lược giữ nguyên vẹn mỗi mục AST thành một `chunk` tạo ra phân phối độ dài dạng heavy-tailed với P95 đạt 2.518–3.112 ký tự và cực đại 15.726 ký tự, vượt quá context window 256 token của mô hình Cross-Encoder hiện tại nên bước reranking chỉ quan sát được phần đầu của các mục quá dài; và (3) Cấu hình FTS `simple` không tự động thực hiện sub-token splitting cho các định danh ghép dạng `camelCase`/`PascalCase` trong `tsvector` index, đồng thời quy tắc `Sparse Match Gating` có thể gán nhãn `low` cho các query hợp lệ nhưng không có lexical overlap.
- **Hướng phát triển:** (1) Xây dựng công cụ tự động khép kín vòng lặp để `Coding Agent` tự lấy mẫu một kho tài liệu mới bất kỳ, sinh code `Parser`, chạy bộ test 5 invariants và tự sửa lỗi cho đến khi nạp thành công toàn bộ kho dữ liệu vào framework; (2) Thay thế tầng Cross-Encoder 256 token bằng mô hình Long-Context Reranker hoặc cơ chế chấm điểm trượt theo sliding window để bao phủ toàn bộ phần thân của các mục tài liệu dài trên 2.500 ký tự; và (3) Bổ sung bước sub-token splitting cho compound identifiers trước khi đưa vào `tsvector` index cũng như hỗ trợ trích xuất tự động edge candidates cho Knowledge Graph trong pha

ingestion.

TÀI LIỆU THAM KHẢO

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[3] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.

[4] Y. Zhang *et al.*, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," *arXiv preprint arXiv:2506.05176*, 2025.

[5] A. Kusupati *et al.*, "Matryoshka representation learning," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30233–30249, 2022.

[6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, pp. 758–759, 2009.

[7] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[8] W. Wang *et al.*, "MiniLMv2: Multi-head self-attention relation distillation for compressing pretrained transformers," in *Findings of the Association for Computational Linguistics: ACL-IJCNLP*, pp. 2140–2151, 2021.

[9] S. Zhou *et al.*, "DocPrompting: Generating code by retrieving the docs," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[10] Z. T. Wang *et al.*, "CodeRAG-Bench: Can retrieval augment code generation?," in *Findings of the Association for Computational Linguistics: NAACL*, 2025.

[11] Y. Zhang *et al.*, "cAST: Enhancing code retrieval-augmented generation with structural chunking via abstract syntax tree," *arXiv preprint arXiv:2506.15655*, 2025.

[12] Anthropic, "Introducing Contextual Retrieval," Anthropic Engineering Technical Report, Sep. 2024.

[13] D. Edge *et al.*, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[14] Anthropic, "Model Context Protocol (MCP) Specification," https://modelcontextprotocol.io, 2024.

PHỤ LỤC

- **Kho mã nguồn mở của dự án:** https://github.com/**[<tên-nhóm>/<tên-dự-án>]** cung cấp toàn bộ mã nguồn, PostgreSQL/pgvector database schema, benchmark corpus và hướng dẫn reproduce môi trường qua Docker Compose và trình quản lý gói `uv`.
- **Đặc tả API của 6 MCP tools chuẩn [14] (hỗ trợ cả `stdio` và Warm Streamable HTTP `POST /api/mcp`):**
  1. `hybrid_search(query, pattern, limit, path_prefix, doc_slugs, is_regex, case_sensitive, rerank)`: Thực thi dual-channel retrieval kết hợp Hybrid RRF + Cross-Encoder Reranking với verbatim/regex matching, trả về các context groups đã sắp xếp theo reading order và confidence label γ.
  2. `hierarchical_navigate(path, direction, limit)`: Duyệt cây phân cấp `ltree` theo 4 hướng `CHILDREN`, `PARENT_CHAIN`, `SIBLINGS`, `SUBTREE` kèm cơ chế tự động tái tạo các virtual parent nodes khi mục cha chỉ đóng vai trò container.
  3. `graph_traverse(source_path, direction, max_depth, filter_relations, limit_per_link)`: Duyệt đệ quy Knowledge Graph theo hướng `OUTGOING`, `INCOMING`, `BOTH`, tự động xử lý các quan hệ đối xứng và phân bổ quota kết quả công bằng cho từng liên kết.
  4. `graph_curate_edges(reason, add_edges, remove_edges)`: Thêm hoặc xóa trong một atomic transaction các graph edges kèm kiểm chứng whitespace-invariant citation grounding, mở rộng container target node xuống các leaf nodes và ghi append-only audit log.
  5. `get_raw(path_or_slug, start_line, end_line)`: Trích xuất nguyên văn line slice $[s, e]$ với giới hạn tối đa 200 dòng mỗi lần gọi trực tiếp từ raw text của tài liệu.
  6. `submit_grounded_answer(answer, citations, abstained)`: Xác thực câu trả lời của `Agent` bằng cách kiểm tra từng trích dẫn `(path, start_line, end_line)` trực tiếp trên PostgreSQL, trích xuất văn bản nguyên gốc và từ chối ngay lập tức nếu đường dẫn `ltree` không tồn tại trong kho ngữ liệu.
- **Giao diện Corpus & Graph Observatory Dashboard (`uv run rag-eval ui`):** Cung cấp cây thư mục tài liệu và `ltree` trực quan, chế độ xem đối chiếu raw text theo dòng, bản đồ Knowledge Graph 2D hiển thị in-degree/out-degree của từng node, giao diện thử nghiệm Venn retrieval và không gian **Hỏi đáp có kiểm chứng trích dẫn (Grounded Agent Q&A)** hiển thị trực tiếp tiến trình gọi công cụ MCP theo thời gian thực qua WebSocket (`/api/qa/ws`) cùng thẻ đối chiếu văn bản nguyên gốc cho từng dấu trích dẫn `[#k]`.
- **Mẫu các ca kiểm thử tiêu biểu từ bộ dữ liệu `data/eval/retrieval.jsonl`:** Bảng 16 trích dẫn 6 ca kiểm thử đại diện cho hai bộ `TS1-Lexical-API` và `TS2-Conceptual` trên hai miền Next.js và Node.js, minh họa rõ vai trò của từng phân vùng Venn (`both`, `semantic_only`, `verbatim_only`).

Bảng 16. Mẫu 6 truy vấn kiểm thử tiêu biểu trong bộ đánh giá 100 câu

Mã	Bộ / Miền	Câu hỏi tóm tắt	query / pattern	Đường dẫn ltree đích	Giỏ / Hạng
e001	TS1 Next.js	Cờ --yes của create-next-app cấu hình mặc định import alias và bundler nào?	create-next-app --yes... pattern: "--yes"	nextjs.01_getting_started.01_installation.quick_start	both (#1)
e021	TS1 Next.js	Truyền FormData của Server Action vào Object.fromEntries sinh ra các thuộc tính nội bộ có tiền tố gì?	Server Action FormData... pattern: "$ACTION"	nextjs.02_guides.forms.how_it_works	verbatim(#1)
e029	TS1 Node.js	Trên Windows, khi options.env của spawn chứa cả khóa PATH và Path, Node.js chọn khóa nào?	child_process spawn... pattern: "options.env"	nodejs.child_process.overview	both (#1)
e053	TS2 Next.js	Vì sao triển khai nhiều replica Next.js tự host gây lỗi Failed to find Server Action và cần biến môi trường nào?	self-hosting multiple... pattern: "NEXT_SERVER_..."	nextjs.02_guides.self_hosting.multi_server_deployments.server_functions_encryption_key	both (#1)
e072	TS2 Next.js	Trong kiến trúc Multi-Zones, vì sao liên kết giữa các zone phải dùng thẻ <a> và tránh đụng độ tài nguyên tĩnh thế nào?	Multi-Zones links... pattern: "assetPrefix"	nextjs.02_guides.multi_zones.linking_between_zones	semantic(#2)
e083	TS2 Node.js	Sau khi monkey-patch CommonJS export của node:fs, làm sao đồng bộ lại các named exports bên ES module?	syncBuiltinESM... pattern: "syncBuiltin..."	nodejs.module.the_module_object.module_syncbuiltinesmexports	verbatim(#1)

GIẢNG VIÊN HƯỚNG DẪN
Ký và ghi rõ họ tên

.....

ĐẠI DIỆN NHÓM THỰC HIỆN
Ký và ghi rõ họ tên

.....